

Efficient Reasoning on the Edge

Yelysei Bondarenko*, Thomas Hehn*, Rob Hesselink*, Romain Lepert*, Fabio Valerio Massoli*, Evgeny Mironov*, Leyla Mirvakhabova*, Tribhuvanesh Orekondy*, Spyridon Stasis*, Andrey Kuzmin†, Anna Kuzina†, Markus Nagel†, Ankita Nayak†, Corrado Rainone†, Ork de Rooij†, Paul N Whatmough†, Arash Behboodi‡, Babak Ehteshami Bejnordi‡

Large language models (LLMs) with chain-of-thought reasoning achieve state-of-the-art performance across complex problem-solving tasks, but their verbose reasoning traces and large context requirements make them impractical for edge deployment. These challenges include high token generation costs, large KV-cache footprints, and inefficiencies when distilling reasoning capabilities into smaller models for mobile devices. Existing approaches often rely on distilling reasoning traces from larger models into smaller models, which are verbose and stylistically redundant, undesirable for on-device inference. In this work, we propose a lightweight approach to enable reasoning in small LLMs using LoRA adapters combined with supervised fine-tuning. We further introduce budget forcing via reinforcement learning on these adapters, significantly reducing response length with minimal accuracy loss. To address memory-bound decoding, we exploit parallel test-time scaling, improving accuracy at minor latency increase. Finally, we present a dynamic adapter-switching mechanism that activates reasoning only when needed and a KV-cache sharing strategy during prompt encoding, reducing time-to-first-token for on-device inference. Experiments on Qwen2.5-7B demonstrate that our method achieves efficient, accurate reasoning under strict resource constraints, making LLM reasoning practical for mobile scenarios. Videos demonstrating our solution running on mobile devices are available on our project page:

<https://qualcomm-ai-research.github.io/llm-reasoning-on-edge/>.

1 Introduction

The success and impact of large language models (LLMs) continue to expand, with reasoning emerging as a fundamental component of their achievements. Commercial-grade coding agents reason about code structure, refactoring, debugging, and dependency resolution [1; 2]. In scientific domains, LLMs are increasingly used to assist professional mathematicians with research level problems [3; 4; 5]. On mobile devices, reasoning-capable LLMs unlock a new class of intelligent personal assistants able to plan multi-step tasks, respond contextually to user queries, and operate autonomously across apps and interfaces. Recent advances from Gemini and OpenAI demonstrate increasingly capable standalone reasoning [3; 6], and a growing wave of agentic use cases is now emerging where models interact directly with device UIs and real-world services [7; 8; 9; 10]. These achievements, however, come at the cost of generating massive numbers of tokens, with reasoning traces accounting for a large fraction of the overall computation [11].

Deploying reasoning models on edge devices is attractive for mobile scenarios because it keeps sensitive data on-device, reduces round-trip latency, and remains available even under limited connectivity. In practice, however,

*Core contributors

†Contributors

‡Project leads

Names within each group are sorted alphabetically.

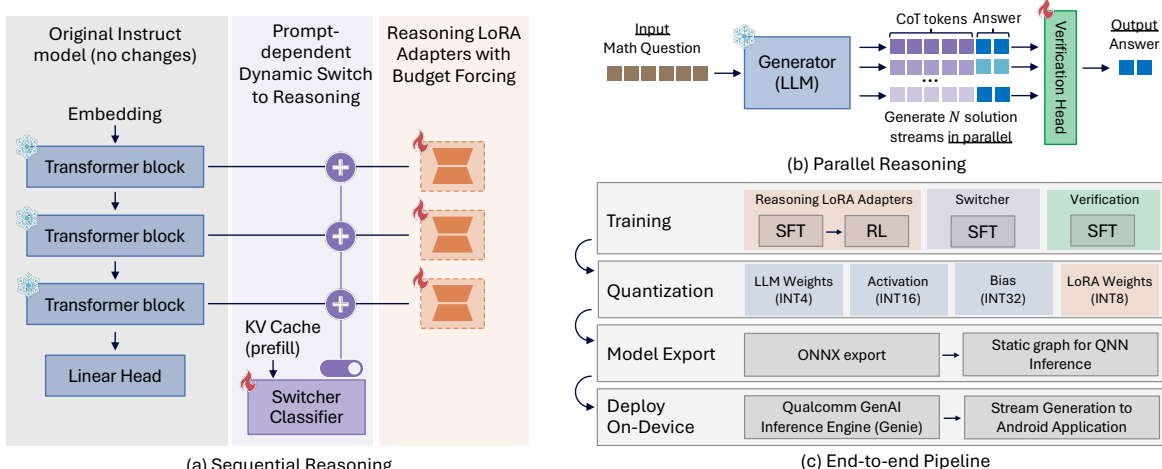


Figure 1: **Overview of the proposed efficient reasoning framework for edge devices.** (a) The model architecture utilizes parameter-efficient LoRA adapters and a lightweight switcher to dynamically route queries. This design allows the base model and the reasoning-activated mode to seamlessly share a reusable KV cache during prefill. (b) Parallel test-time scaling strategy, generating multiple reasoning streams concurrently to improve accuracy without severe latency penalties. (c) The end-to-end deployment pipeline, illustrating the progression from multi-stage training (SFT and budget-forced RL) to quantization, model export, and final on-device execution.

on-device reasoning faces several key limitations. The first is the memory bottleneck: mobile devices, constrained by current DRAM capacities, can typically support smaller models with moderate quantization, and larger models only with more aggressive quantization schemes [12; 13]. The second limitation is the cost of token generation in terms of power consumption, latency, and memory footprint. Long reasoning traces and large context lengths substantially increase KV cache size, quickly exhausting available memory. Finally, general purpose LLMs with broad capability scopes are difficult to realize within the model sizes supported by edge devices. While specialized small language models (SLMs) can match the performance of larger models on targeted tasks, model switching introduces additional memory movement overhead. Edge deployed models must therefore operate within strict memory budgets while achieving usable tokens per second (TPS) and acceptable time to response.

This work proposes an end-to-end pipeline for deploying reasoning-capable language models on edge devices under strict token, latency, and memory budgets. Our design starts from a base non-reasoning instruct model and enables a “reasoning mode” via LoRA adapters, so the same backbone can run either in standard chat mode (no adapters) or in reasoning mode (reasoning adapters enabled). A lightweight switcher routes each incoming query to the appropriate mode, enabling reasoning only when it is likely to help.

At training time, we use parameter-efficient fine-tuning to specialize the base model across domains while keeping deployment practical. We demonstrate that LoRA [14] is effective for enabling domain targeted reasoning and task specialization. Selecting the LoRA rank as a function of the base model and desired performance is a central question, which we address through extensive analysis. Because LoRA adapters can be toggled at runtime, a single base model can be loaded once and then dynamically adapted to different tasks by enabling or disabling adapters. To enable KV-cache sharing between the base model and the LoRA augmented reasoning model, we propose masked LoRA training during the prefill phase, which we show has no significant impact on accuracy.

Our training recipe follows a two-stage structure commonly used for reasoning models: supervised fine-tuning (SFT) on high-quality reasoning traces, followed by a reinforcement learning (RL) phase for further alignment [15]. Since, reasoning verbosity is not explicitly addressed during SFT and models often become verbose and repetitive after initial training, a key objective of our RL phase is to penalize excessively long reasoning traces via budget forcing [16; 17]. We apply budget forcing during RL and explore different training strategies, reward designs, prompting

methods, and hard enforcement mechanisms, distilling best practices from our experiments.

At inference time, we target memory-bound decoding as a primary on-device bottleneck. We leverage parallel test-time scaling with neural verification to improve accuracy without incurring significant latency overhead, particularly in typical on-device settings. Concretely, because on-device inference splits into a compute-bound prefill phase and a memory-bound decoding phase, we can better utilize compute units (e.g., NPUs) by running parallel decoding paths with minimal incremental overhead. We further show that neural verification in an outcome reward model style can be implemented using a lightweight verifier head trained on the latent representations of the base model.

Together, these components allow us to start from a non-reasoning model and progressively harvest reasoning performance while maintaining deployability on edge devices. Figure 1 provides a comprehensive overview of our proposed end-to-end framework, illustrating the complete progression from adapter training to on-device deployment. We instantiate this pipeline on Qwen2.5 series of models, and we enable reasoning for these models using our proposed pipeline. On device deployment requires additional considerations. We discuss a range of quantization schemes supporting 4- to 8-bit weight quantization while allowing higher precision for activations to accommodate their dynamic range, minimizing quantization loss throughout the pipeline. The resulting quantized models are then exported and compiled for on-device execution. The entire workflow is implemented using Qualcomm open source tooling, including Qualcomm FastForward [18] and Qualcomm GENIE SDK [19]. In this paper, we provide deeper insights into the design choices and empirical analyses underpinning this work. We position the resulting system as a practical blueprint for deploying reasoning capable language models on resource constrained edge devices.

2 Reasoning on Edge: System Design

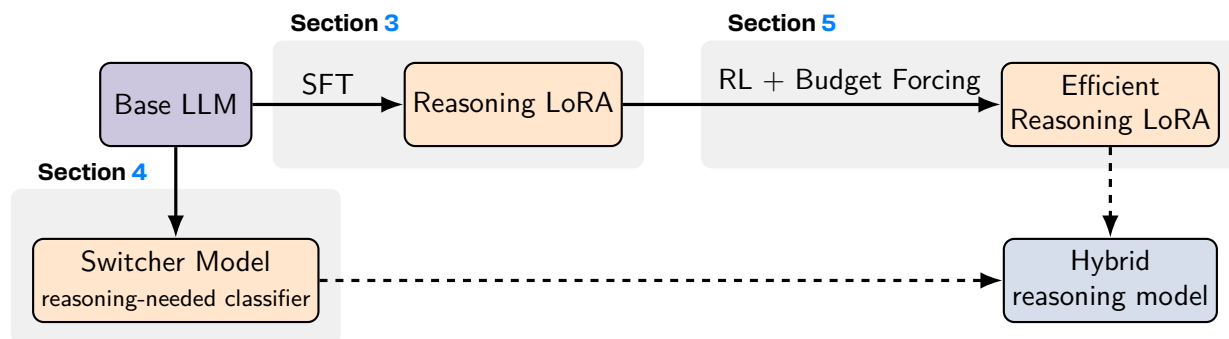


Figure 2: **Architecture of the Hybrid Reasoning Model.** The pipeline begins with a compact base LLM, which is specialized for reasoning via LoRA-based supervised fine-tuning (SFT). To enforce concise generation and prevent excessive verbosity, these adapters undergo reinforcement learning (RL) with Budget Forcing. Finally, a lightweight Switcher module is introduced to act as a reasoning-needed classifier, creating a hybrid model that dynamically routes incoming queries to either the fast base model or the specialized reasoning adapters based on task complexity.

Developing compact yet capable reasoning models requires a training pipeline that can reliably elicit high-quality chain-of-thought (CoT) behavior from a relatively small base LLM while avoiding unnecessary verbosity and excessive computation. Although pretrained LLMs can sometimes function as zero-shot reasoners [20], explicit reasoning via approaches such as scratchpads [21] or supervised CoT [22] remains essential for strong performance on mathematics and coding tasks. Recent systems such as OpenAI O1 [2] and specialized small reasoning models, including Tina [23], Phi-4-mini [24], and hybrid reasoning architectures [25], illustrate that such capabilities can be distilled into relatively small models when combined with targeted fine-tuning and alignment techniques.

Our objective is to fine-tune a modest base LLM so that it performs competitively on complex reasoning tasks

while producing concise outputs and minimizing token generation. To keep adaptation lightweight and deployable, we perform training primarily through Low-Rank Adapters (LoRA) [14], which preserve a reusable frozen backbone and enable modular reasoning specialization. The overall pipeline, presented in Figure 2, integrates supervised fine-tuning, reinforcement learning, and lightweight routing in a manner that preserves efficiency and supports deployment under strict memory and latency constraints.

Supervised fine-tuning constitutes the first stage of our pipeline and is designed to unlock the reasoning capabilities of the pretrained base LLM. Rather than performing dense fine-tuning, we adopt parameter-efficient fine-tuning using LoRA. LoRA has been shown to match or even surpass dense fine-tuning in reasoning settings [26], while enabling the base model to remain frozen and reusable for multiple domains. This stage equips the model with the fundamental ability to reason through multi-step problems, but, as widely observed, also increases verbosity and can lead to unnecessarily long or repetitive traces [15].

To refine the reasoning behavior and control verbosity, we apply **Reinforcement Learning** [27; 28] (RL) using a custom reward function tailored to two objectives: accuracy and efficiency. First, we use budget forcing [29], a mechanism that penalizes excessively long responses. This constraint encourages the model to produce concise reasoning traces without sacrificing correctness. Second, we incorporate an answer-based reward, which directly incentivizes the model to generate correct final answers. For optimization, we employ the group-based relative policy optimization (GRPO) algorithm [30], which updates the LoRA parameters.

Finally, not all user queries require multi-step reasoning. To avoid unnecessary computation, we introduce a lightweight **Switcher module** that predicts whether reasoning is needed based on hidden prompt representations. When reasoning is unnecessary, the model bypasses the LoRA adapters and relies on the base model directly, reducing latency and limiting KV cache growth, an important consideration for edge deployment.

In the following sections, we decompose our end-to-end pipeline into its main components, LoRA-based adaptation (Section 3), dynamic inference-time routing using a Switcher module to activate or bypass these adapters (Section 4), budget-forced RL for verbosity control (Section 5), and parallel test-time scaling (Section 6), and the deployment path including quantization and on-device execution (Section 7). In each section, we describe the component and then quantify its impact with targeted experiments, highlighting key insights and focused ablations to isolate which design choices drive accuracy and which improve on-device efficiency.

3 LoRA for Modular Reasoning

In our reasoning framework, we adopt parameter-efficient fine-tuning (PEFT) for two reasons: first, it enables scalable experimentation at low training costs, and second, it produces modular adapters that can be enabled or disabled at runtime, allowing a single base model to switch between general-purpose chat and enhanced reasoning modes. To elicit reasoning behavior, we perform SFT on datasets composed of reasoning traces generated by stronger teacher models such as DeepSeek-R1 [15] and QwQ-32B [31].

Our experiments show that 3B and 7B models can acquire strong reasoning ability using straightforward SFT on curated trace datasets in a cost-efficient setup, reaching performance comparable to substantially larger distilled baselines (e.g., DeepSeek-R1-Distill-Qwen-7B). These results suggest that strong reasoning does not require heavy distillation pipelines or large training budgets; high-quality trace data (e.g., OpenThoughts3 [32]) combined with lightweight fine-tuning is sufficient to close most of the gap. Overall, this provides a practical and scalable path to building capable reasoning models without expensive infrastructure.

3.1 Experimental Setup

The primary goal of our LoRA adaptation stage is to determine if small, general-purpose instruct models can acquire expert-level reasoning capabilities without full-parameter distillation. Accordingly, we adopt the Qwen2.5-

3B-Instruct and Qwen2.5-7B-Instruct models [33] as our core experimental backbones. In this section, we outline the datasets and optimization strategies used to elicit reasoning behavior via LoRA, as well as the diverse suite of math, science, and coding benchmarks used to rigorously evaluate the resulting performance trade-offs. We additionally provide an extensive LoRA hyperparameter study designed to identify the most stable and compute-efficient adaptation strategies for both the 3B and 7B backbones.

3.2 Training Details

Data. In our experiments, we utilize two SFT datasets. The first is **Mixture of Thoughts** (MoT) [34], which contains 350k reasoning traces distilled from the DeepSeek-R1 model. This dataset covers three core domains: Math (93.7k traces), Code (83.1k traces), and Science (173k traces). The second dataset is **OpenThoughts3-1.2M** (OT3) [32], comprising 850k Math questions, 250k Code questions, and 100k Science questions. The annotation traces for OT3 were generated using the QwQ-32B model.

Training configuration. All models were trained for 5 epochs using the bfloat16 data type with the DeepSpeed zero2 configuration and CPU offloading enabled. Across all configurations, we applied a cosine learning rate schedule with a warmup ratio of 0.1, and weight decay was set to 0.

For the baseline dense training on the MoT dataset, the learning rate was set to $1e-5$ for Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct models, with a global batch size of 128. Model weights were optimized using the AdamW optimizer with $(\beta_1, \beta_2) = (0.9, 0.95)$. For dense training on the OT3 dataset, we followed the recipe described in [32]. Dense models were trained with a learning rate of $8e-5$ and a batch size of 512, using AdamW with $(\beta_1, \beta_2) = (0.9, 0.999)$.

In our LoRA training setup, we follow the common practice of employing relatively larger learning rates. We also find that reducing the batch size generally leads to more stable optimization and improved results. Throughout all PEFT experiments, we use a LoRA rank of 128, set the LoRA alpha to twice this value, adopt a learning rate of $2e-4$, and train with a batch size of 64.

3.3 Evaluation Details

Benchmarks. We assess the reasoning capabilities of our trained models using a diverse set of benchmarks that span tasks across mathematics, science, and coding domains. To evaluate multi-step mathematical problem-solving, we use challenging competition datasets including AIME 24/25 [35], AMC23 [36], and MATH500 [37]. Scientific reasoning is measured using the PhD-level GPQA Diamond dataset [38]. Finally, to evaluate code generation, we utilize LiveCodeBench (v2, code generation scenario only) [39] for recent competitive programming problems, alongside standard Python programming tasks from HumanEval [40] and MBPP [41], including their rigorously verified EvalPlus variants (HumanEval+ and MBPP+) [42]. Comprehensive descriptions of each benchmark, including problem counts and specific testing scenarios, are provided in Appendix A.

Evaluation pipeline. Our evaluation pipeline is structured as follows. For reasoning benchmarks including AIME24, AIME25, MATH500, GPQA, and AMC, we allow a generation length of up to 32,768 tokens, with the temperature set to 0.6 and top_p to 0.95. For models trained on the OT3 dataset, we adopt the generation parameters recommended by the authors in [32], specifically setting the temperature to 0.7 and top_p to 1.0 and keeping generation length up to 32,768 tokens. Also for LCB, MBPP and HumanEval (and their + counterparts) we set the generation parameters as recommended by the models’ creators; we set the maximal generation length to 32768 tokens for LCB, and 1024 tokens for HumanEval and MBPP.

Given that some benchmarks contain a limited number of questions and are therefore more susceptible to accuracy variance, we perform multiple evaluation runs to ensure robustness similarly to the protocol outlined in [32]. Specifically, we evaluate the AIME24, AIME25 and AMC datasets 10 times and report the averaged results. For

GPQA, we conduct 4 evaluation runs and for MATH500 we run the evaluation once. For coding benchmarks, the pass@1 score is estimated from a pool of 16 candidate solutions for LCB, and of 200 candidate solutions for HumanEval and MBPP, using the unbiased pass@ k estimator first proposed in [40]. All evaluations are conducted using the lighteval framework [43] with vLLM support [44], except for those on HumanEval, MBPP and their enhanced variants, for which we used the Evalplus package¹.

Table 1: Results of the main experiments. FT data indicates the finetuning dataset. The LoRA column specifies the adapter rank, or its absence for dense finetuning. LCB refers to LiveCodeBench, and HE denotes HumanEval. R1* corresponds to the R1-Distill-Qwen-7B model released by DeepSeek; we evaluated this model following our standardized evaluation protocol.

	FT data	LoRA	AIME24	AIME25	MATH500	GPQA	AMC23	LCB	HE	HE+	MBPP	MBPP+
Qwen-3B	-	-	0.09	0.03	0.66	0.31	0.36	0.23	0.40	0.64	0.72	0.61
	MoT	-	0.20	0.22	0.80	0.37	0.61	0.38	0.43	0.38	0.34	0.29
	OT3	-	0.51	0.43	0.92	0.40	0.83	0.50	0.48	0.43	0.36	0.30
	OT3	rk 128	0.18	0.21	0.79	0.32	0.55	0.23	0.40	0.35	0.39	0.32
Qwen-7B	-	-	0.10	0.17	0.76	0.37	0.60	0.36	0.83	0.75	0.80	0.68
	R1*	-	0.55	0.40	0.92	0.49	0.89	0.59	0.41	0.38	0.42	0.36
	MoT	-	0.37	0.28	0.90	0.48	0.78	0.55	0.53	0.47	0.60	0.50
	OT3	-	0.61	0.54	0.95	0.47	0.89	0.66	0.58	0.51	0.56	0.46
	OT3	rk 64	0.47	0.39	0.92	0.45	0.78	0.48	0.60	0.54	0.63	0.52
	OT3	rk 128	0.56	0.38	0.93	0.43	0.82	0.54	0.60	0.54	0.57	0.46
	OT3 + MoT	rk 128	0.44	0.39	0.93	0.48	0.83	0.51	0.59	0.52	0.60	0.49

3.4 Results

As shown in Table 1, we evaluate Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct models under dense and LoRA-based fine-tuning, and compare against key baselines. Finetuning on the OpenThoughts3 (OT3) dataset yields the largest and most consistent gains in reasoning performance across both backbones, improving accuracy substantially on math and science benchmarks and also boosting performance on the more reasoning-sensitive coding benchmark (LiveCodeBench). In contrast, Mixture of Thoughts (MoT) provides clear improvements over the base models, but its gains are consistently smaller than OT3. In addition, the densely trained Qwen2.5-3B model on OT3 performs on par with, or slightly above, the densely trained Qwen2.5-7B model on MoT, suggesting that higher data quality can partially compensate for smaller backbone size.

For Qwen2.5-7B, LoRA fine-tuning on OT3 recovers most of the dense OT3 improvements, and increasing the adapter rank from 64 to 128 generally narrows the gap on core reasoning benchmarks (e.g., AIME24 and LCB). Notably, OT3 with LoRA rank 128, which requires updating only 4.24% of the parameters compared to dense fine-tuning, reaches performance close to the R1-Distill-Qwen-7B baseline on several reasoning benchmarks, indicating that lightweight adapter training can recover much of a distilled model’s capability at significantly lower adaptation cost. For Qwen2.5-3B, however, OT3 LoRA (rank 128) underperforms the dense OT3 model by a large margin across reasoning benchmarks, highlighting that adapter capacity and/or optimization details matter more at smaller scales and motivating our ablations in the subsequent sections.

The coding results reveal a trade-off between reasoning specialization and “direct-answer” code generation. While SFT consistently improves performance on LCB, it always results in some degradation for MBPP, HumanEval and their respective .+ variants. This pattern is consistent with a shift toward explicit multi-step reasoning: it helps on harder, reasoning-sensitive coding tasks like LCB, but can be counterproductive on benchmarks that reward short, direct code outputs. Concretely, our SFT stage is designed to elicit reasoning behavior into an otherwise

¹<https://github.com/evalplus/evalplus>

non-reasoning model, and LCB is a setting where such explicit reasoning can be beneficial. In contrast, MBPP and HumanEval typically do not prompt the model to reason, and a response must be given directly². Interestingly, for Qwen2.5-7B, OT3 LoRA (rank 64/128) often retains stronger HumanEval/MBPP performance than dense OT3, suggesting that PEFT can partially mitigate the specialization/forgetting trade-off relative to dense fine-tuning. Prior work has similarly observed [45] that SFT on reasoning traces can result in partial forgetting of general capabilities, and that this phenomenon could potentially be mitigated by employing RL fine-tuning [46].

We also explored a two-stage training strategy for the 7B model to see if we could combine the distinct benefits of both datasets. Our motivation was that while the OT3 dataset (generated by the smaller QwQ model) provided the strongest baseline performance, the MoT dataset might contain complementary, highly complex reasoning patterns that the model could absorb in a subsequent training phase. However, interestingly, additional training on MoT following training on OT3 results in minimal changes across benchmarks. Accuracies remain largely stable, except for a 0.12-point decrease on AIME24 and a 0.05-point improvement on GPQA.

3.4.1 LoRA Hyperparameter Study: Rank, Learning Rate, and Batch Size

To study the effect of hyperparameters on the performance of PEFT models, we consider a range of values for learning rate, batch size, and LoRA rank. We trained Qwen2.5-3B-Instruct and Qwen2.5-7B-Instruct on a subset of OT3 consisting of 50,000 entries for 1 epoch. For each model, we vary the values within the following ranges: learning rate in $\{1e-4, 2e-4, 5e-4\}$, batch size in $\{32, 64, 128\}$, and LoRA rank in $\{32, 64, 128, 256\}$. LoRA adapters were applied to all linear layers, with α set to $2 \times \text{rank}$ and dropout fixed at 0.1. For our ablation study, we evaluate the trained models on MATH and Science benchmarks including AIME24, AIME25, MATH500, GPQA Diamond and AMC23. Similarly to the setup used in [32], we average the accuracy values over 10 runs for smaller datasets like AIME24, AIME25 and AMC23, over 4 runs for GPQA and once for MATH500. We set the temperature to 0.7 and *top_p* value to 1.0. Full evaluation results for this ablation study can be found in Appendix B.

Qwen2.5-3B-Instruct results. The full performance of the models trained on a set of considered hyperparameters is summarized in Table 12 in the Appendix. To isolate the effect of each hyperparameter, we summarize results by averaging accuracy over the other two dimensions (learning rate, batch size, and LoRA rank), highlighting the main trends. As shown in Table 2, learning rate has a noticeable impact on performance, with $2e-4$ providing the highest overall average accuracy across tasks. We also observe opposite sensitivities across benchmarks: AIME24 and AIME25 improve as the learning rate increases, whereas MATH500 degrades, suggesting that larger learning rates can lead to over-adaptation on more complex mathematical reasoning.

Table 2: Average performance grouped by learning rate for Qwen2.5-3B-Instruct.

LR	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
1e-4	0.040	0.025	0.573	0.277	0.281	0.239
2e-4	0.051	0.033	0.574	0.274	0.299	0.246
5e-4	0.051	0.038	0.556	0.280	0.275	0.240

As presented in Table 3, LoRA rank shows the most consistent positive influence on aggregate performance. Increasing rank generally improves accuracy, with rank 256 achieving the highest overall average. However, rank 128 already performs strongly on most benchmarks, making it a practical operating point when adapter memory is constrained. Across tasks, AIME25 is particularly sensitive to rank, while MATH500 remains comparatively stable. Overall, higher ranks are preferable when resources permit, but rank 128 offers a favorable accuracy-efficiency trade-off for edge deployment.

²We have nonetheless confirmed that the model’s answers are never cut short by the Evalplus harness.

Table 3: Average performance grouped by LoRA rank for Qwen2.5-3B-Instruct. %TP denotes the percentage of trainable parameters.

Rank	%TP	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
32	1.94%	0.047	0.020	0.568	0.270	0.284	0.238
64	3.88%	0.040	0.023	0.570	0.283	0.284	0.240
128	7.76%	0.054	0.038	0.560	0.272	0.287	0.242
256	15.52%	0.048	0.048	0.573	0.282	0.284	0.247

Finally, batch size had a relatively minor effect compared to learning rate and rank as shown in Table 4. The best overall performance was observed at batch size set to 64, though the differences were small. MATH500 benefited slightly from larger batches like 128, while AIME25 peaked at 64. These results indicate that batch size can be chosen primarily based on computational constraints without significant impact on overall performance.

Table 4: Average performance grouped by batch size for Qwen2.5-3B-Instruct.

Batch size	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
32	0.054	0.030	0.562	0.277	0.288	0.242
64	0.048	0.038	0.566	0.278	0.287	0.243
128	0.040	0.028	0.575	0.277	0.281	0.240

Qwen2.5-7B-Instruct results. Table 13 in the Appendix summarizes the Qwen2.5-7B-Instruct LoRA ablations over learning rate, batch size, and adapter rank. In contrast to the 3B setting, where learning rate affects performance but the averages vary only modestly across the tested values, the 7B backbone exhibits a narrower stable learning-rate range (Table 5): $LR=1e-4$ and $2e-4$ trains reliably, whereas $LR=5e-4$ is often unstable and can lead to collapsed runs. Overall, a practical guideline is to use a lower learning rate for stable training on larger backbones, and tune the remaining hyperparameters within that stable regime. For the individual tables presented below, we account for this distinction explicitly. The table reporting averages across different learning rate settings includes all runs, highlighting the instability introduced by higher learning rates. In contrast, the tables reporting averaged accuracies for batch size and LoRA rank exclude runs with a learning rate of $5e-4$, as those configurations contain diverged results.

Table 5: Average performance grouped by learning rate for Qwen2.5-7B-Instruct.

LR	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
$1e-4$	0.158	0.132	0.775	0.366	0.533	0.393
$2e-4$	0.170	0.148	0.779	0.354	0.543	0.399
$5e-4$	0.148	0.112	0.635	0.350	0.439	0.337

As shown in Table 6, LoRA rank has a measurable but relatively small impact on Qwen2.5-7B performance: the average score improves from 0.388 (rank 32) to 0.402 (rank 128), while rank 256 is comparable at 0.397. Overall, ranks 64–128 form a tight trade-off region, with rank 128 best on average but only marginally better than lower ranks. Compared to the 3B model, the 7B results are more tightly clustered across ranks, indicating that rank is a weaker lever here than it is for smaller backbones.

Similarly, Table 7 shows that batch size has a negligible effect on Qwen2.5-7B performance in our sweep. The average accuracy varies only from 0.394 to 0.398. This mirrors the 3B trend, suggesting batch size can largely be chosen based on computational constraints once learning rate is in a stable regime.

Table 6: Average performance grouped by LoRA rank for Qwen2.5-7B-Instruct. %TP denotes the percentage of trainable parameters.

Rank	%TP	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
32	1.06%	0.152	0.119	0.776	0.357	0.534	0.388
64	2.12%	0.159	0.150	0.779	0.355	0.535	0.396
128	4.24%	0.178	0.141	0.784	0.367	0.541	0.402
256	8.48%	0.165	0.152	0.771	0.359	0.536	0.397

Table 7: Average performance grouped by batch size for Qwen2.5-7B-Instruct.

Batch size	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
32	0.154	0.148	0.781	0.355	0.530	0.394
64	0.164	0.146	0.774	0.366	0.538	0.398
128	0.172	0.127	0.776	0.358	0.541	0.395

As a result of the hyperparameter study, it follows that the setup with learning rate value of $2e-4$, batch size of 64, and LoRA rank set to 128 results in consistently good results while providing a good compromise between efficiency and training stability.

4 Dynamic LoRA Routing via the Switcher Module

While reasoning models excel at complex problem-solving, not every user query requires an exhaustive, multi-step CoT. For standard conversational prompts or straightforward factual questions, generating long reasoning traces incurs unnecessary latency and computational overhead which can be a critical bottleneck for edge devices. To address this, we introduce a lightweight *Switcher* module that enables dynamic adapter routing. By analyzing the user prompt, the switcher decides whether to bypass or activate the reasoning-specific LoRA adapters. When disabled, the system operates as the highly efficient original instruct model for regular conversation; when activated, it seamlessly transitions into a specialized reasoning engine.

Architecturally, the switcher serves as an auxiliary classification head on top of the base LLM and operates during the prefilling stage. It processes the hidden states from the final transformer layer and computes an averaged sequence representation. Based on this representation, the switcher performs binary classification to determine whether the input sequence corresponds to a reasoning-oriented task. If classified as such, the reasoning LoRA adapters are activated for subsequent decoding.

The switcher head is implemented as a lightweight multilayer perceptron (MLP) with a single hidden dimension of 8, a ReLU activation function, and a dropout rate of $p = 0.2$. This compact architecture ensures negligible overhead for efficient on-device inference while maintaining sufficient expressive capacity for sequence-level classification.

On edge devices, the prefill phase is heavily compute-bound. Processing a long input prompt in a single pass can incur prohibitive computational overhead. To mitigate this, practical on-device implementations typically divide the prefill sequence into smaller, discrete chunks. The switcher module is explicitly designed to support this chunked prefill strategy. Rather than buffering the hidden states of the entire prompt to compute a global average, the switcher updates its sequence representation on the fly. Specifically, we compute a running exponential moving average of the hidden states across these chunks. In our setup, we use a chunk size of 128 tokens with a smoothing coefficient of $\alpha = 0.5$. To enhance robustness to quantization artifacts, we inject independent Gaussian noise with zero mean and standard deviation $\sigma = 0.5$ into the averaged representation during training.

Masked LoRA training for KV-cache reuse. A major challenge with dynamically activating LoRA adapters at inference time is KV-cache compatibility. Under standard LoRA training, the model expects the KV cache for the prompt tokens (the prefill phase) to be generated with the LoRA adapters fully active. If a query is routed to the reasoning mode after the base model has already encoded the prompt, the system would typically need to re-encode the entire prompt with the LoRA adapters activated to generate a compatible KV cache. On edge devices, this re-encoding incurs a severe latency and compute penalty. To eliminate this inefficiency, we introduce a *masked LoRA* training strategy. During the fine-tuning of the reasoning adapters, we mask (disable) the LoRA weights during the forward pass of the prompt tokens, activating them only for the generation of the response tokens. This forces the LoRA adapters to adapt to the prompt KV cache generated strictly by the base model. Empirically, we observe that this strategy incurs no drop in reasoning accuracy, while allowing the base model and reasoning mode to seamlessly share a single prefill KV cache, entirely obviating the need to re-encode prompt tokens when switching.

4.1 Training details

To train the switcher, we constructed a small dataset that combines both straightforward conversational or knowledge based queries and complex queries, enabling the model to learn how to distinguish when reasoning is required. The dataset is structured as follows: each entry consists of a question prompt paired with a label 0 or 1, indicating low or high complexity, respectively. Complexity is determined based on the source of the dataset from which the prompt was sampled. We included prompts from datasets covering math and non-math domains to reduce the risk that the switcher overfits to domain-specific cues (e.g., assuming all math questions are inherently more difficult).

The final dataset contains approximately 2k samples. Easy queries were randomly drawn from the SQuAD2.0 dataset (600 questions) [47], which consists primarily of general-knowledge comprehension questions, and from the MMLU math subset (419 questions) [37], which includes straightforward mathematical problems. Hard prompts were sourced from a subset of the S1K dataset (500 questions) [16], encompassing challenging questions spanning math, science, and crosswords, as well as from StrategyQA (500 questions) [48] covering reasoning questions from non-scientific domains.

4.2 Results

The primary motivation for the switcher module is to optimize standard, day-to-day user interactions. In real-world edge deployments, the vast majority of user queries are simple conversations, factual lookups, or basic instructions that do not require multi-step reasoning. By aggressively thresholding the switcher to route these simple queries to the base instruct model, we can achieve massive aggregate savings in token generation, latency, and power consumption, reserving the LoRA reasoning adapters strictly for complex tasks.

To rigorously assess the impact of this dynamic routing, we evaluated the switcher’s performance on the challenging MATH500 benchmark as it contains questions with varying complexity. We swept across different switcher confidence thresholds to vary the fraction of prompts routed to the reasoning adapters versus the base model. Figure 3 illustrates how overall model performance changes as a larger fraction of queries is routed to the reasoning model. Here, we study the base Qwen2.5-7B-Instruct model and its counterpart trained on OT3 with LoRA adapters of rank 128 without any budget forcing.

As more answers are generated with reasoning, accuracy rises smoothly from the base model baseline toward the reasoning-only upper bound. This demonstrates that the switcher effectively prioritizes the reasoning model on more complicated queries where it is most beneficial, allowing accuracy levels that cannot be achieved by the base model alone.

The right panel shows the corresponding computational cost, measured as average completion length. Choos-

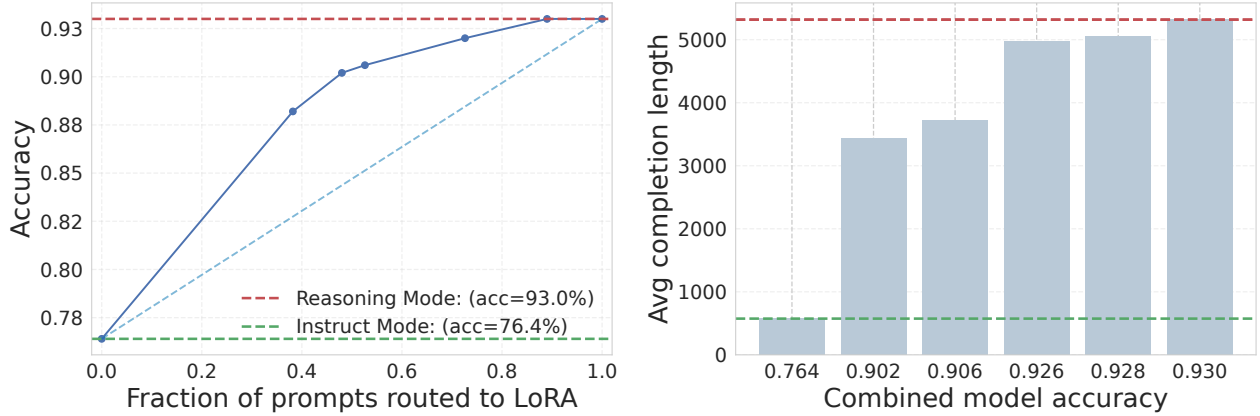


Figure 3: **Impact of the Switcher module on MATH500. Left:** Combined model accuracy as the fraction of queries routed to the reasoning adapters. **Right:** Average completion length versus overall accuracy across different switcher thresholds.

ing a higher-accuracy operating model requires a proportional increase in computational costs, while lower-cost regimes are possible when accuracy demands are modest. The switcher thus provides a flexible mechanism for navigating this tradeoff.

5 Budget Forcing and Inference-Time Compute Optimization

CoT prompting [22] scales Inference-Time Compute (ITC) by decoding intermediate reasoning steps, substantially improving LLM performance on complex tasks, but often at the cost of high latency and large token footprints. Theoretical analysis [49] argue that the optimal test-time compute should scale linearly with problem difficulty. Yet unconstrained models routinely violate this optimality, exhibiting degenerate verbosity and overthinking even on trivial tasks [16].

To reconcile reasoning capability with computational efficiency, *Budget Forcing* [16] methods aim to align generation with explicit token/compute constraints. RL-based methods, among others [50; 51; 52; 53], achieved impressive trade-off between performance and CoT length reduction. These methods typically require to augment the reward with a length-based penalty term [54] or to enforces hard truncation constraints upon reaching a target budget [55]. Formally, a standard budget-forced reward objective can be expressed as an additive penalty:

$$R(y, x) = R_{\text{accuracy}}(y, x) - \lambda \cdot R_{\text{budget}}(L) \quad (1)$$

where x is the prompt, y is the generated response, $R_{\text{accuracy}}(y, x)$ is the accuracy reward, L represents the total token length, and $R_{\text{budget}}(L)$ is a penalty function scaled by the hyperparameter λ .

5.1 Soft-Barrier Reward Formulation

Building upon the foundations of budget-penalized RL, our reward is based on three core rationales:

1. **Avoidance of strict token matching:** We do not force the model to exactly match a predefined budget. Doing so assumes perfect a priori knowledge of the optimal compute required for a specific task, which contradicts the premise of generative exploration.
2. **Trajectory exploration:** The model must retain sufficient degrees of freedom to explore diverse reasoning paths without premature truncation.

3. **Prompt-adherent budget compliance:** The model must reliably satisfy the user-defined budget constraints provided in the prompt.

To realize these principles, we prompt the model with discrete generation length budgets, specifically bucketing constraints into 1000, 3000, 4000, and 6000 tokens. Instead of an additive penalty, we introduce a multiplicative, piecewise-linear *soft barrier*. This barrier decays the budget reward from 1.0 to 0.0 as the generation length exceeds the prompted bucket. The linear decay serves as a buffer, discouraging the model from exceeding the limit without inflicting catastrophic penalties for minor budget infractions.

This decay operates within a symmetric window centered around the target budget B , where the half-size of the window, $m \in [0, 1]$, is treated as a tunable hyperparameter. Formally, we define the budget reward modifier as:

$$R_{\text{budget}}(L) = \begin{cases} 1 & L \leq L_{\text{low}} \\ p & L > L_{\text{high}} \\ 1 - (1 - p) \frac{L - L_{\text{low}}}{L_{\text{high}} - L_{\text{low}}} & L_{\text{low}} < L \leq L_{\text{high}} \end{cases} \quad (2)$$

where L is the total length of the generated response, p is the maximum budget penalty floor, $L_{\text{low}} = (1 - m)B$, and $L_{\text{high}} = (1 + m)B$. Through empirical observation, we noted that setting a negative penalty floor provided no optimization benefits; thus, we set $p = 0$.

The final holistic reward R is then defined as the product of the task accuracy and the budget compliance modifier:

$$R(y, x) = R_{\text{accuracy}}(y, x) \times R_{\text{budget}}(L) \quad (3)$$

where $R_{\text{accuracy}}(y, x) \in \{0, 1\}$ represents the binary accuracy reward.

Challenges and reward hacking. Because budget forcing imposes a strict constraint on the optimization manifold, it inherently induces a trade-off between reasoning performance and compute cost. We observed that naively applying a length penalty (for instance, penalizing *only* the tokens within the CoT reasoning trace) is highly susceptible to reward hacking. During early iterations, the policy rapidly collapses into a degenerate, “lazy” strategy: the model learns to circumvent the penalty by prematurely closing the reasoning block with a `</think>` token, only to continue its verbose CoT in the final response output. By penalizing the total generation length L rather than just the reasoning trace, our multiplicative formulation effectively neutralizes this exploit. Furthermore, while our final reward formulation strips away explicit format-following rewards, empirical evaluations confirm that the model consistently maintains the desired structural formatting throughout training.

5.2 Experimental Setup

5.2.1 Training Details

To demonstrate the efficacy of our soft-barrier reward formulation in compressing CoT reasoning, we conduct extensive experiments on state-of-the-art reasoning models. We utilize the DeepScaleR dataset [56] as our primary training corpus. To maximize training stability and prevent degenerate optimization steps, we apply a rigorous filtering criterion to the dataset: any prompt exhibiting a group reward standard deviation of zero is removed, ensuring that the model always receives a meaningful comparative signal during policy updates.

We optimize our models using GRPO [30]. GRPO is particularly well-suited for reasoning tasks as it bypasses the need for a separate value model by leveraging group-scaled rewards. For a given prompt x , the objective minimizes the following loss:

$$\mathcal{L}_{\text{GRPO}}(\theta \mid x) = -\frac{1}{G} \sum_{i=1}^G \min\left(\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i\right) + \beta D_{\text{KL}}(\pi_{\theta}(\cdot \mid x) \parallel \pi_{\text{ref}}(\cdot \mid x)), \quad (4)$$

where G denotes the group size, and the probability ratio ρ_i and the advantage A_i are defined as:

$$\rho_i = \frac{\pi_\theta(y_i | x)}{\pi_{\text{old}}(y_i | x)}, \quad A_i = \frac{r_i - \mu_r}{\sigma_r + \varepsilon} \quad (5)$$

Here, μ_r and σ_r represent the mean and standard deviation of the rewards within the group, respectively:

$$\mu_r = \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_r)^2} \quad (6)$$

Our implementation is built on the **trl** library (version 0.26.2) [57]. We execute training on a single compute node equipped with 8 NVIDIA H100 (80GB) GPUs. We sample 8 generations per prompt ($G = 8$) during the GRPO rollouts. A comprehensive summary of the training hyperparameters is detailed in Table 14 in Appendix C.

5.2.2 Evaluation Details

To assess the impact of our budget forcing technique on mathematical reasoning, we use the large-scale Math500 [58] benchmark as our main testbed. For robust and reproducible evaluation, we employ the **lighteval** framework (version 0.8.1). All inference processes are accelerated using vLLM (version 0.10.2). To standardize the assessment of pass@1 accuracy, we apply a consistent sampling strategy across all benchmarks: generations are sampled with a temperature of 0.6, a top_p of 0.95, and a maximum completion length extended to 32K tokens to accommodate any lingering verbose trajectories from the baseline models.

5.3 Results: Efficiency-Accuracy Trade-off

As established, our primary objective is to compress the generated reasoning trajectories with minimal degradation in task performance. Because our soft-barrier reward formulation deliberately omits a strict regularizer weight for the budget penalty (see eq. 1), we discovered that the Kullback-Leibler (KL) divergence penalty coefficient in GRPO (β_{KL}) serves as an effective control mechanism to enforce budget-friendly behavior. Empirically, setting $\beta_{\text{KL}} = 10^{-3}$ yields the optimal balance, significantly reducing generation length with negligible performance drops. Conversely, a relaxed penalty of $\beta_{\text{KL}} = 10^{-4}$ improves formatting adherence at very short completion lengths, albeit at the cost of a slightly higher performance regression when evaluated on larger, unbounded contexts.

Figure 4 illustrates the average completion length distributions for the unconstrained baseline (purple) and two intermediate checkpoints trained with $\beta_{\text{KL}} = 10^{-3}$. The left and right panels depict evaluations where the maximum completion length is strictly capped at 4K and 6K tokens, respectively. To enforce this hard budget during inference, we abruptly truncate the generation upon hitting the token limit and subsequently append a prompt forcing the model to immediately output its final answer. As demonstrated in Figure 4, our RL fine-tuning effectively shifts the distribution density toward significantly shorter lengths. Crucially, the transition from the baseline (purple) through the intermediate checkpoint (blue) to the final policy (green) highlights a stable, progressive optimization trajectory. Rather than experiencing sudden, erratic policy collapse, the model smoothly and monotonically learns to generate more concise reasoning traces over the course of training to solve the tasks.

To quantify this compression, Figure 5 provides a granular breakdown of the actual CoT length reductions achieved via our RL fine-tuning. Specifically, the right panel of Figure 5 reveals that our approach yields an average completion length reduction factor of $\sim 2.4\times$, with maximum compression rates reaching up to $\sim 8\times$ on certain queries. As noted previously, this aggressive reduction in verbosity is achieved while maintaining comparable performance to the base model, with only minimal-and in many instances, negligible-accuracy drops. Practically, this reduction directly translates to lower overall inference latency and a faster time-to-final-answer, making advanced reasoning models significantly more viable for deployment in resource-constrained environments.

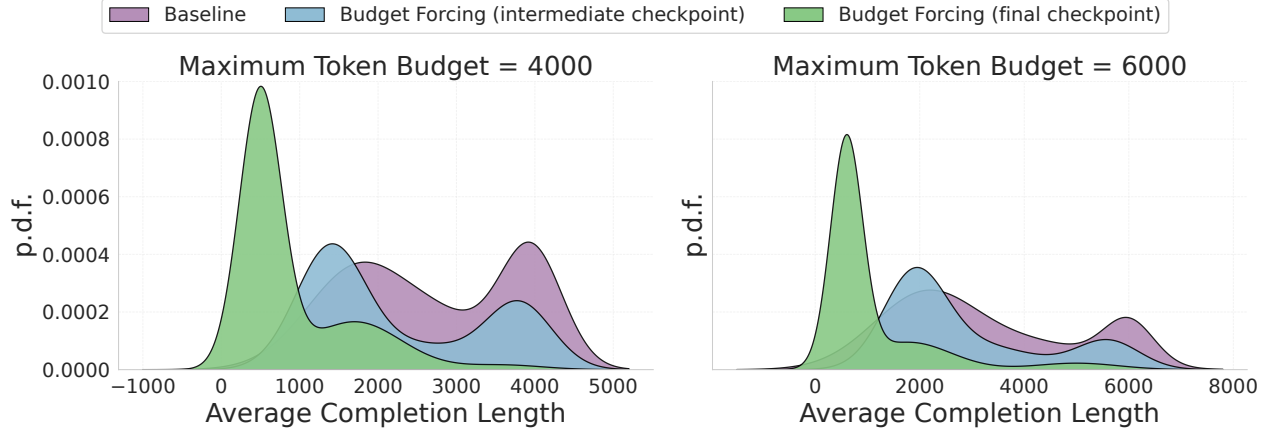


Figure 4: **Average Completion Length Distributions.** **Left:** Evaluation with a forced maximum completion length of 4K tokens. **Right:** Evaluation with a maximum of 6K tokens. Note that distribution tails extending below zero or above the maximum budget are standard artifacts of Kernel Density Estimation (KDE) curve smoothing. The progression from the baseline (purple) through the intermediate (blue) to the final RL fine-tuned checkpoint (green) demonstrates stable, progressive learning of concise generation ($\beta_{KL} = 10^{-3}$).

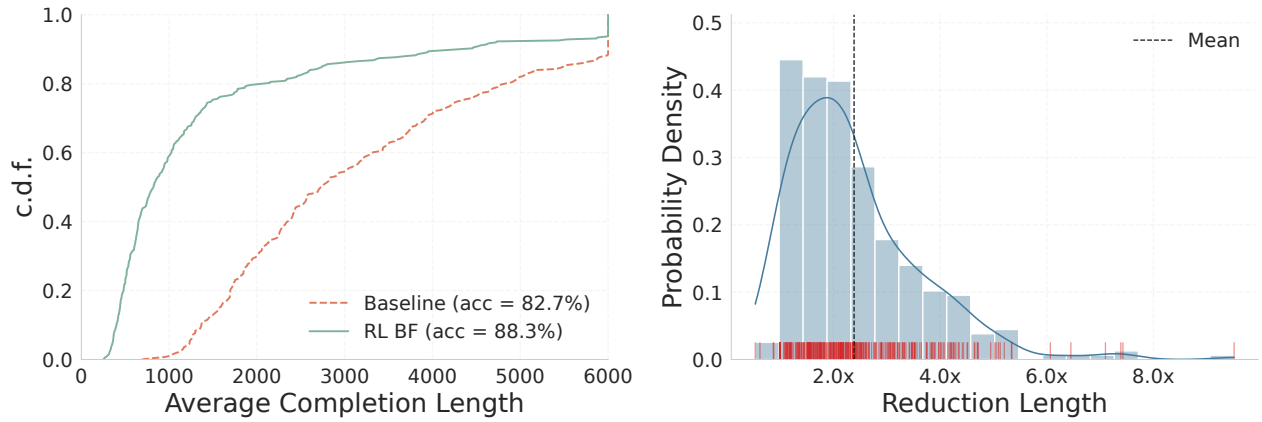


Figure 5: **Average Completion Length Comparison.** **Left:** C.D.F. of the average completion length for base model, orange curve, and RL fine-tuned one, green curve, with $\beta_{KL} = 1.e^{-3}$. We considered a maximum completion length of 6K. **Right:** Reduction in completion length from the RL fine-tuned model. We use the same models in the left plot. The RL fine tuned achieved average reduction length of 2.38 ± 0.07 .

Table 8: **Performance Results on MATH500.** Results for different budget values. For budgets $\in \{1K, 2K, 4K, 6K\}$, we manually force the model to generate the final answer once the limit is hit. In the last column, we do not force the model to generate the answer. We let it generate tokens until the budget hit.

Model	Accuracy $\uparrow$ (%)				
	Budget = 1K	Budget = 2K	Budget = 4K	Budget = 6K	Average (Budget = 32K)
SFT Baseline (r=128)	34	57	73	83	95
BF RL $\beta_{KL}=1.e^{-3}$	<u>62</u>	<u>78</u>	85	90	<u>92</u>
BF RL $\beta_{KL}=1.e^{-4}$	72	80	<u>84</u>	<u>85</u>	90

5.4 Qualitative Analysis of Budget-Forced CoT

To examine the mechanics of our budget forcing objective at the trajectory level, we qualitatively compare the reasoning traces of the unconstrained baseline against our budget-forced model across four distinct mathematical

Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

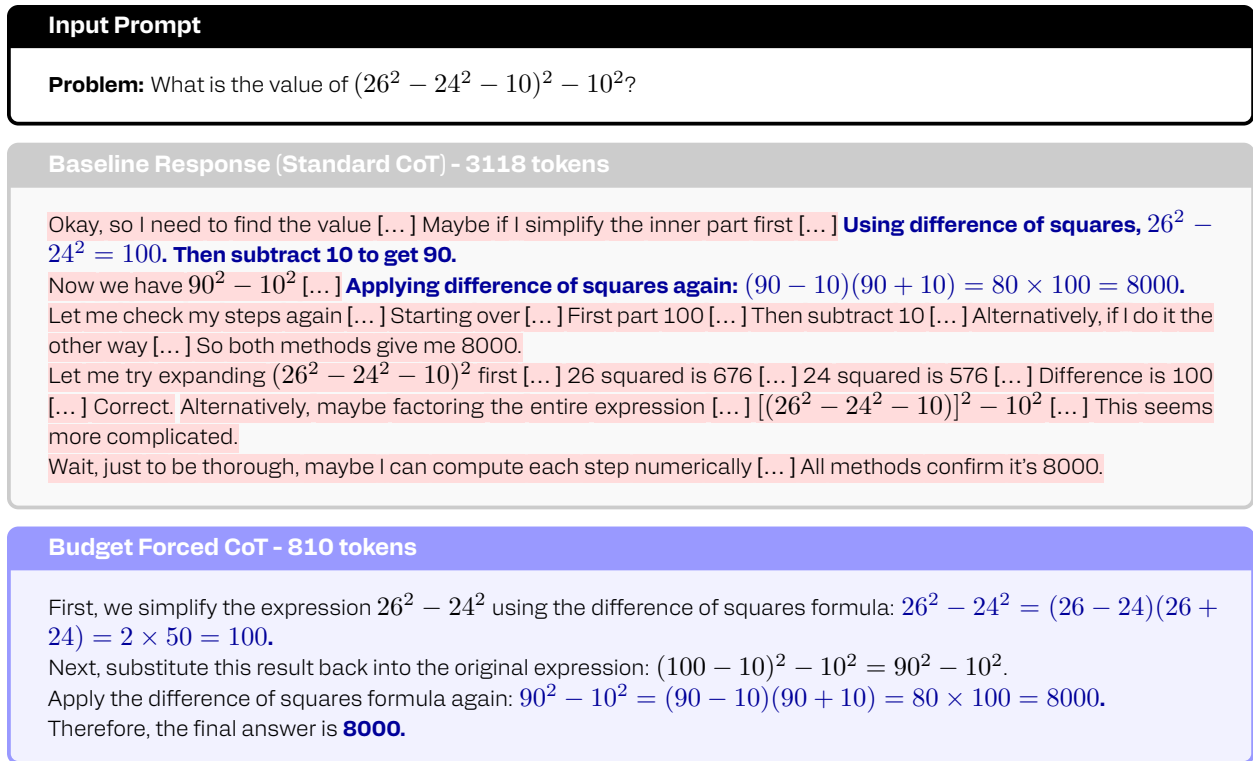


Figure 6: **Qualitative comparison on algebraic simplification. Middle:** The Baseline trace correctly identifies the difference of squares strategy immediately but engages in excessive self-verification, re-calculating the result via expansion, direct computation, and alternative factorizations. **Bottom:** The Budget Forced trace recognizes the nested difference of squares structure and executes the solution linearly without redundant checking.

domains: number theory, algebraic simplification, pattern recognition, and modular arithmetic (Figures 6–7 and Figures 9–10 in Appendix C). A consistent pattern emerges from this analysis: the unconstrained baseline frequently suffers from severe epistemic hesitation. While it typically identifies the correct logical strategy early in the generation process, it expends thousands of tokens on redundant self-verification, testing alternative (and often less efficient) methods, and hypothesizing trivial errors. For instance, in Figure 6 and Figure 7, the baseline arrives at the correct answer almost immediately but falls into extensive validation loops, re-calculating the result using three to four different approaches.

In contrast, our budget-forced policy learns to confidently trust its initial, correct logical derivations. It successfully prunes these redundant validation loops and verbose syntactic parsing, drastically reducing the trace length while strictly preserving the essential reasoning backbone and human readability. Furthermore, although we omit special delimiter tokens (e.g., `<think>`) in the visualizations for brevity, we observe that the budget-forced model robustly generates them, maintaining the required CoT and final-answer formatting constraints.

6 Parallel Test-Time Scaling and Reasoning

During autoregressive generation with LLMs, one of the dominant runtime bottlenecks is repeatedly loading model weights of the individual layers to produce the next token. As a result, there is often potential to increase the total amount of computation performed per user query without incurring significant overhead in runtime. A straightforward way to do this is to generate multiple independent samples in parallel: instead of producing one CoT trajectory and one final answer, this approach allocates additional compute to produce several trajectories concurrently.

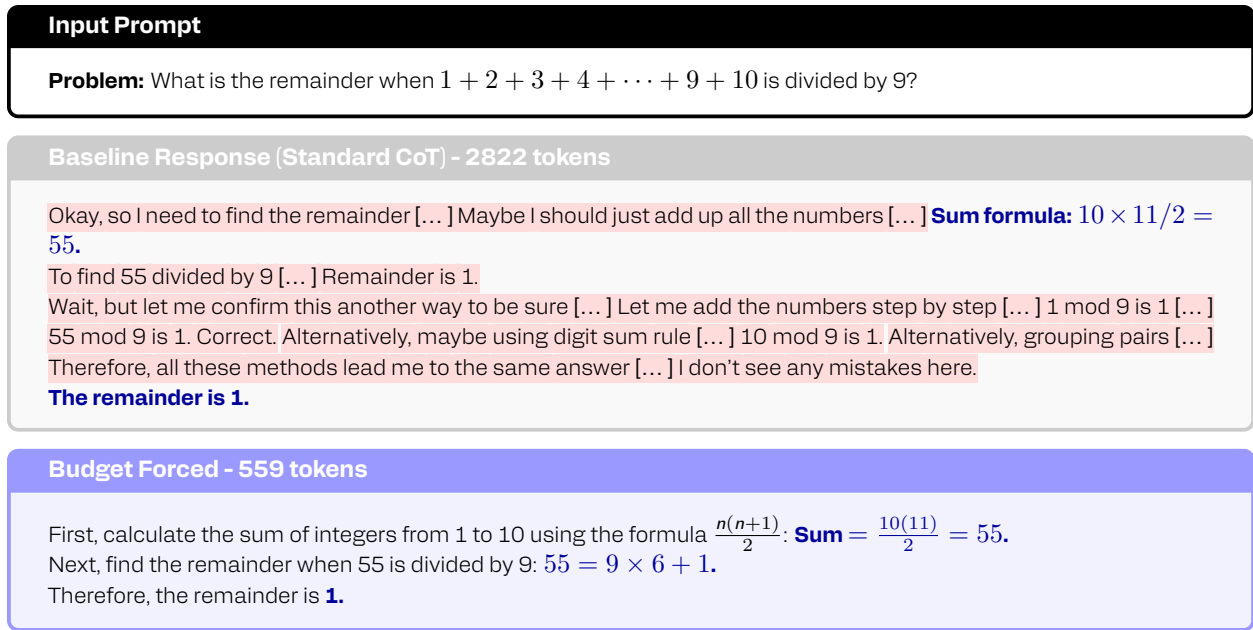


Figure 7: **Qualitative comparison on modular arithmetic. Middle:** The Baseline trace correctly computes the sum and remainder immediately but engages in extensive, redundant verification using four different methods (step-by-step addition, digit sum rule, pairing, and re-calculation). **Bottom:** The Budget Forced trace performs the direct calculation and returns the result without hesitation.

Parallel generation is not only attractive from a systems perspective, but it also improves accuracy. Across benchmarks and models, generating multiple candidate solutions and then aggregating them has repeatedly shown consistent performance gains. A common and simple aggregation strategy is majority voting over individual answers, where the most frequently produced answer is selected as the final answer. In practice, majority voting is widely used as a reliable tool for achieving an additional boost in final model performance from the same underlying base model.

Initial works [59; 60] indicated that the same LLM can be made to generate multiple diverse and independent responses towards finding a better solution. Scaling compute by sampling multiple independent responses prior to aggregation has shown [61; 62; 63] to be an effective paradigm enabling smaller models to outperform larger models for the same inference compute budget. In these cases, the key is to score the solutions (e.g., by frequency, or using an external verifier) and predict the highest-scoring solution. Parallel reasoning design relies on selecting a sampling scheme, design and employ reward models, and finally aggregation scheme to generate the final answer. Diversity among responses [61] is crucial to increasing the probability of sampling the correct response. Increasing diversity is typically done by auto-regressive generation at a higher sampling temperature [64]. While responses are typically sampled independently, recent works explore inter-dependent sampling [65; 66; 67; 68] and guided-search [69; 70; 71] towards generating a better response candidate pool. In this work, we focus on independent sampling schemes. Reward models (also referred to as verifiers) estimate one or more scalar-valued scores for a given response. There is a long line of work on improving reward models, with more accurate reward models [72; 73], more granular score assignment (e.g., per-step scores with process reward models) [58; 74; 75], scaling verification with more compute [76], and extending reward models to other reasoning domains [77; 78] beyond mathematical reasoning. In this work, we use the reward models for scoring the outcome. Finally, aggregation is essential to drafting a specific response from a pool of multiple response-score pairs. Existing works have primarily looked at this from a rank-and-select lens: to rank the responses either by frequency of occurrence ('self-consistency' or 'majority voting') [60], or using external reward models to score responses [59; 74]. Note that these strategies lead to a zero-sum situation: the top-ranking solution is selected and the rest discarded. Consequently, a more

recent line of work [79; 80; 81; 82] investigates synthesizing (rather than selecting) a final response based on the candidate responses.

Scaling compute at test-time generally implies additional latency and compute overhead and hence making it especially challenging to realize on resource-constrained edge devices. Some benefits of parallel TTS on edge devices were discussed in [83] with a separate verifier. This separation prevents efficient reuse of intermediate computations, e.g., through the KV-cache. Consequently, scaling parallel compute *efficiently* has gained some attention in the research community. Towards efficiently reasoning, several works have investigated generating solutions by performing fewer FLOPs [62] and reducing the memory footprint [84; 85] required during generation. Closest to our verifier design presented in this paper is GenRM [86], where the authors investigate finetuning the base generative model to additionally perform prompt-based verification. Such a joint generation-verification paradigm is appealing for edge devices, since generation and verification can be performed with minimal movement of parameters between DRAM and flash memory.

6.1 Efficient Verifier Design for Edge Compute

Parallel reasoning produces a set of N independent CoT traces and corresponding final answers. The central question then becomes: given these N candidates, how do we reliably choose the correct one? Majority voting provides a strong baseline, but it is not always sufficient, especially when the answer space is large or ambiguous. This motivates introducing an explicit selection mechanism that can score candidates and prefer those most likely to be correct, while still benefiting from the diversity created by independent sampling.

A natural approach is to add a verifier model that evaluates each candidate solution. However, deploying a separate verifier on edge devices at the same scale as the generator can be prohibitive in storage and memory footprint, and often becomes especially costly in latency. To avoid this, we aim to reuse the generator as effectively as possible. Concretely, we keep the same base model and add a lightweight verifier head: a separate linear layer applied to the final token embedding, followed by a sigmoid activation to yield a scalar “correctness” score. Figure 1b illustrates this approach. This design keeps additional parameters minimal and, crucially, allows KV-cache reuse for all generated responses, since the verifier head can operate on representations already computed during generation.

In addition to the linear head, we append a short verification prompt after each generated response, asking the model whether the proposed solution is correct. Empirically, this extra query has been beneficial compared to relying on a linear head alone without an explicit verification prompt. Operationally, this strategy only requires a small additional prefill step for the verification prompt for each individual generation, while preserving KV-cache reuse from the original generation. The verifier therefore adds only modest overhead per candidate while improving the reliability of the selection process.

To obtain the best results, we combine majority voting with verifier scoring into a weighted majority vote. Instead of counting each candidate answer equally, we weight each candidate’s vote by its verifier score. Intuitively, candidates that the verifier deems more likely to be correct contribute more to the final decision, while still retaining the robustness benefits of aggregation across multiple samples. This hybrid approach preserves the simplicity and stability of voting while incorporating a learned notion of solution quality, which is particularly helpful when the candidate set contains a mix of superficially plausible but incorrect solutions alongside correct ones.

6.2 Training and Evaluation Details

Verification is treated as a binary classification problem where a candidate response of the generator is labeled as correct or incorrect with respect to the ground-truth answer. To generate training data for the verifier, we use 97.5% of the 7.5k questions from the MATH training set [37]. The remaining 2.5% of questions are reserved for validation of the verifier. To construct a diverse training set and a validation set for fast evaluation, we generate 16

candidate responses for each training question, while we limit ourselves to 4 generated responses in the validation set. The verifier head uses a sigmoid activation to produce a probability-like score, and we train it with binary cross-entropy loss. This setup aligns the verifier objective directly with the downstream selection problem: distinguishing correct from incorrect candidate solutions produced by the generator under the same sampling procedure used at inference time.

Table 9: Accuracy of our lightweight verifier weighted majority voting compared to majority voting without verifier and greedy decoding on MATH500. The mean and standard deviation are computed from 20 random draws from 16 independent 4bit-weight-quantized Qwen-2.5-7B-Instruct responses.

Parallel Responses	1	2	4	6	8
Greedy (baseline)	71.0	-	-	-	-
Majority Vote	69.9 ± 1.3	70.0 ± 1.3	75.1 ± 1.0	76.6 ± 1.0	77.5 ± 0.8
Weighted MV (ours)	69.9 ± 1.3	72.7 ± 1.0	76.1 ± 0.9	77.5 ± 0.8	78.2 ± 0.7

6.3 Results

We evaluate the proposed lightweight verifier on MATH500 using a 4-bit-weight-quantized Qwen-2.5-7B-Instruct model, comparing greedy decoding, standard majority voting, and our weighted majority voting. Table 9 summarizes accuracy as a function of the number of parallel responses, sampled with temperature set to 0.7.

Even with very limited parallelism, parallel test-time scaling yields immediate benefits. With just two parallel responses, weighted majority voting improves accuracy to 72.7%, outperforming both the greedy baseline (71.0%) and standard majority voting (70.0%). This highlights a key advantage of incorporating verification: when two sampled responses disagree, majority voting cannot break ties, whereas the verifier-weighted scheme can consistently select the more reliable candidate.

As the degree of parallelism increases, both majority voting and weighted majority voting exhibit steady gains, confirming prior observations that parallel sampling improves the probability of generating a correct solution. However, weighted majority voting consistently outperforms unweighted majority voting across all parallelism levels, and at eight parallel responses, weighted majority voting improves upon the baselines by 10%. Importantly, the variance across random draws is also slightly reduced compared to majority voting, suggesting that verifier weighting provides a more stable aggregation mechanism.

Despite its simplicity, the verifier delivers strong benefits. Architecturally, it amounts to generating only a minimal amount of overhead, effectively one extra token per stream. Because the verifier reuses the generator’s KV-cache, it does not require reprocessing the original prompt or response, avoiding the dominant memory and latency costs typically associated with separate verifier models. This makes the performance gains particularly compelling in edge settings, where memory bandwidth and storage are tightly constrained.

7 Quantization

In the following section, we discuss the details of our quantization methodology. We begin by providing a brief overview of neural network quantization and a summary of recent methods for quantizing LLMs in Section 7.1.

We outline our strategy for quantizing base LLM in Section 7.2 demonstrated on Qwen2.5-7B-Instruct, and later equip it with reasoning capabilities in Section 7.3. Lastly, we provide the specifics of quantized model export and deployment on device in Section 7.4.

7.1 Background and Related Work

Quantization. Neural network quantization is one of the most powerful ways to reduce model footprint, data transfer and compute requirements [87; 88]. By quantizing a model, high bit-width floating point weights and activations can be represented using low-bit numbers. Next to reducing model size, the use of low-bit fixed-point representations, such as INT8, can also significantly reduce the latency and energy consumption [89].

We use the following definition of the quantization-dequantization function:

$$\hat{\mathbf{x}} := q(\mathbf{x}; \mathbf{s}, \mathbf{z}, b) = \mathbf{s} \cdot \underbrace{\left(\text{clip}\left(\left\lfloor \frac{\mathbf{x}}{\mathbf{s}} \right\rfloor + \mathbf{z}; -2^{b-1}, 2^{b-1} - 1\right) \right)}_{=: \mathbf{x}_{\mathbb{Z}}}, \quad (7)$$

where $\mathbf{x}$ denotes the quantizer input (i.e., network weight or activation tensor), $\mathbf{s}$ the high precision (FP32 / FP16 / BF16) quantization scale, $\mathbf{z}$ the integer zero offset, and b the bitwidth. $\lfloor \cdot \rfloor$ denotes the round-to-nearest-integer operator. $\mathbf{x}_{\mathbb{Z}}$ is a b -bit integer *quantized representation* of the input $\mathbf{x}$. Quantization parameters $\mathbf{s}, \mathbf{z}$ can be shared across the components of $\mathbf{x}$ (typically per-channel or block-wise). This quantization scheme is called *uniform affine* or *asymmetric* quantization [90; 87; 91] and is one of the most commonly used quantization schemes because it allows for efficient implementation of fixed-point arithmetic. In the case of *symmetric* quantization, we restrict the quantization grid to be symmetric around $\mathbf{z} = \mathbf{0}$.

Quantization methods can generally be categorized into *post-training quantization* (PTQ) and *quantization-aware training* (QAT) families. PTQ algorithms convert pretrained high-precision networks directly into fixed-point models without the need for the original training pipeline [92; 93; 94; 95; 96; 97; 98; 99; 100]. These approaches are fast, easy to use, and typically rely only on a small calibration dataset. In contrast, QAT methods [101; 102; 103; 104] simulate quantization during training to find more optimal solutions, but generally require longer training, more memory, labeled data, and careful hyperparameter tuning.

LLM Quantization. The excessive training cost and memory usage of traditional QAT methods renders them less practical for quantizing modern LLMs, although some works such as LLM-QAT [105] and BitDistiller [106] explore QAT with knowledge distillation. Notably, [107; 108] are the only studies we are aware of that successfully scale QAT to billions of tokens. Several papers explored the combination of QAT and parameter-efficient fine-tuning (PEFT), including [109; 110; 111; 112; 113; 114]. Most of these approaches offer a substantial memory reduction compared to traditional QAT, but generally are not focused on inference efficiency. For instance, QLoRA [109] quantizes the pretrained weights to 4 bit using (a non-uniform) NF4 format but dequantizes them in the forward pass back to BF16.

Post-training quantization of LLMs is a challenging task due to presence of strong numerical outliers in weights and activations [115; 116; 117; 118; 119]. The core challenge is that quantizing outliers onto a fixed-point grid forces a range-precision trade-off: increasing the dynamic range captures outliers but sacrifices precision near zero, while retaining precision requires clipping them – both of which strongly degrade model performance.

Existing LLM PTQ methods can be broadly categorized into *weights-only* quantization and *weight-activation* quantization. Weights-only quantization focuses on converting weights to low-bit values. GPTQ [120] employs second-order information to iteratively round grouped weights and correct the quantization error in the remaining groups. SpQR [121], AWQ [122] and OWQ [123] emphasize the importance of so-called “salient” weights that correspond to high-magnitude activations. Other recent W-only methods include [124; 125; 126; 127]. Weight-activation quantization compresses both weights and activations. SmoothQuant [128], LLM.int8() / GPT3.int8() [117] and Outlier Suppression [129] achieve W&A quantization by managing activation outliers. LLM.int8() uses mixed-precision decomposition, while the other two employ channel-wise scaling. Some of the other recent W&A PTQ methods are [130; 131; 132; 133; 134; 135; 136].

LLM Quantization using FPTs. A promising direction in LLM quantization is the use of *rotations* and

other *function-preserving transformations* (FPTs). Nagel et al. [98] first explored FPTs for CNN quantization, showing that ReLU and per-channel scaling commute, enabling cross-layer rescaling of weights. In the LLM setting, Xiao et al. [128] propose migrating problematic outliers from the activations into the weights through online per-channel scaling applied before linear layers. Follow-up work extends this idea by incorporating shifts into the scaling [132], scaling vectors for queries and keys [137], channel-mixing transforms [127], randomized Hadamard transforms to reduce outliers [138; 139], other online rotations [140], combinations of scaling and rotations [141], and Kronecker-structured matrix transforms [142].

Recently, FPTQuant [143] introduced three novel, lightweight, and expressive FPTs to facilitate quantization of transformers. By leveraging the equivariances and independencies inherent to modern transformers, these FPTs are designed to maintain the model’s function while shaping the intermediate activation distributions to be more quantization friendly. As a result, FPTQuant enables static INT4 quantization with virtually no overhead and no custom kernels, is very fast and performs on par or exceeds most prior work.

7.2 Quantizing Base Language Model

Quantization setup. To run the model efficiently on our hardware, we quantize the weights of all linear layers including the final LM head using INT4 per-channel uniform affine quantization (eq. 7). To further improve efficiency and reduce latency, we use INT8 KV-cache, INT8 input embeddings and INT16 for all remaining activations (all per-tensor). For brevity, we will refer to this configuration as ‘W4A16KV8’. We use symmetric quantization for weights, KV-cache and embeddings, and asymmetric quantization for activations, which is a common setting.

Transformations. To maximize the accuracy of the quantized model, we apply the subset of fully-mergeable transformations from FPTQuant [143] (Figure 8):

- a pair of pre-RoPE transforms $(\mathbf{T}_k, \bar{\mathbf{T}}_k)$, where $\mathbf{T}_k$ is applied to keys and $\bar{\mathbf{T}}_k$ can be interpreted as an inverse of $\mathbf{T}_k$, applied to the queries;
- $(\mathbf{T}_u, \mathbf{T}_u^{-1})$ a per-channel scaler merged into up and down projection weights;
- multi-head value transforms $(\mathbf{T}_v, \bar{\mathbf{T}}_v)$, which consist of invertible matrices per head merged into value and output weights;
- and a rotation matrix $(\mathbf{T}_r, \mathbf{T}_r^{-1})$ for rotating the residuals (applied at the beginning and the very end of each transformer block and is shared).

This set of transformations $\mathbf{T} := \{\mathbf{T}_k, \mathbf{T}_u, \mathbf{T}_v, \mathbf{T}_r\}$ will help shape the intermediate activation distributions to be more quantization friendly, while keeping the unquantized model outputs intact. All of the above FPTs are fully mergeable, so that we can run the model without any extra inference overhead on our hardware.

Training and evaluation details. Following literature [143; 144], we train the model on DCLM-Edu [145], a cleaner filtered version of DCLM [146] obtained by applying an educational quality classifier [147]. We initialize quantization parameters by minimizing the L^p ($p = 2$) norm between quantized and unquantized tensors, and then train transformation $\mathbf{T}$ and quantization parameters $\{\mathbf{s}, \mathbf{z}\}$ end-to-end, closely following the pipeline of FPTQuant. We simulate quantization using FastForward [18]. For brevity, we denote to the aforementioned quantization pipeline as **FPTQuant^o**.

To assess the predictive performance of the quantized base language model, we follow the previous work [120; 128; 137; 143; 142], and report WikiText-2 test perplexity (assuming a sequence length of 4096). We also report an average zero-shot accuracy of a set of common sense reasoning (CSR) tasks, that includes PIQA [148], Wino-Grande [149], HellaSwag [150], ARC-e and ARC-c [151], and LAMBADA [152]. Finally, we report 5-shot accuracy on MMLU [153]. For CSR and MMLU evaluation, we use the LM Harness framework [154].

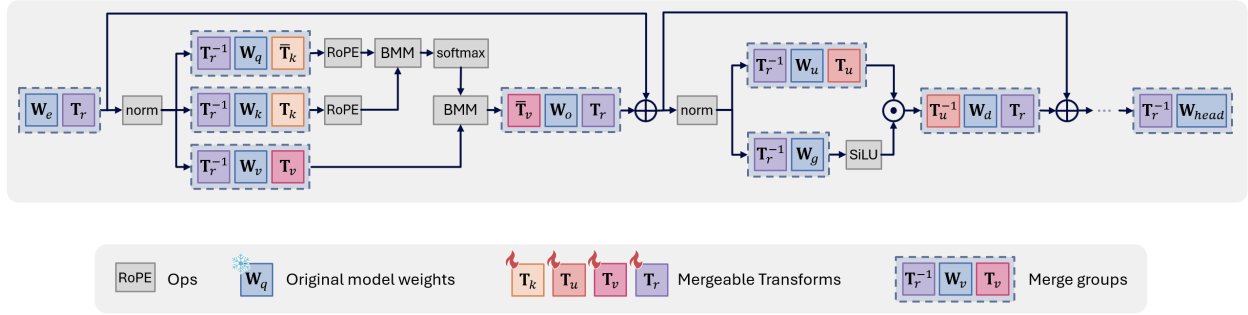


Figure 8: **Function-Preserving Transformations.** We use 4 transform types from FPTQuant: scale-and-rotate transform $\mathbf{T}_k$ merged into query and key, a per-channel scaler $\mathbf{T}_u$ merged into up and down projection, and $\mathbf{T}_v$ that consists of invertible matrices per head merged into value and output weights, and a rotation matrix $\mathbf{T}_r$ for rotating residuals (shared across layers). After training of the transforms is complete, the transformation parameters from each ‘merge group’ are merged into the original model weights $\mathbf{W}$.

Table 10: **Quantized W4A16KV8 base model (Qwen2.5-7B-Instruct) results.** We report Wikitext perplexity, average 0-shot CSR, and 5-shot MMLU accuracies. ‘ L^P ’ denotes the use of L^P range initialization, $\mathbf{T}$ = using the set of mergeable transforms, ‘train’ = end-to-end training of transformation and quantization parameters.

Method	Bitwidth	L^P	$\mathbf{T}$	train	WikiText-2 ($\downarrow$)	CSR ($\uparrow$)	MMLU ($\uparrow$)
Full-precision	BF16	-	-	-	6.85	72.90	74.28
Min-max quantization	W4A16KV8	-	-	-	102.4	51.71	62.35
	W4A16KV8	✓	-	-	9.18	65.83	67.59
	W4A16KV8	-	✓	-	8.48	67.85	69.06
	W4A16KV8	✓	✓	-	7.53	70.68	72.26
FPTQuant^o (ours)	W4A16KV8	✓	✓	✓	7.26	72.94	72.81

Results. We summarize our results for quantized base model in Table 10. As we can see, the simplest PTQ pipeline with min-max range estimation experiences an unacceptable accuracy/perplexity drop. Both strong numerical outliers in the activations (even with 16-bits!) but mainly the catastrophic loss of precision in the quantized 4-bit weights lead to such poor performance.

Employing the set of function-preserving mergeable transformations $\mathbf{T}$ already significantly improves the distribution of weights and activations, leading to much better accuracy, even with min-max range setting. Further, using a better range initialization together with end-to-end learning both progressively recover a greater portion of the full-precision model performance. In the end, we match full-precision accuracy on CSR, and have about 0.4 perplexity drop on WikiText-2 and just under 1.5% accuracy drop on MMLU, where the latter is known to be quite a challenging benchmark. Overall, FPTQuant^o-quantized base model demonstrates strong performance, given that the entire process took less than 24 hours on a single Nvidia H100 80GB GPU.

7.3 Quantization-Aware Modular Reasoning

Quantizing the base model will inevitably affect the underlying activation distributions. To achieve the best performance, it is crucial to account for these changes when applying subsequent fine-tuning, including our Reasoning LoRA protocol described in Section 3.

Our approach follows the general paradigm of QLoRA [109] and related techniques [110; 111], in which LoRA

Table 11: **Quantized W4A16KV8 reasoning model results (Qwen2.5-7B-Instruct base)**. We report AIME24/25, MATH500, GPQA, and AMC23 accuracies (higher is better). N = number of OT data examples used for training reasoning module.

Bitwidth	FPTQuant ^o	QAMR	N	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
BF16	-	-	50k	21.8	20.3	82.6	38.6	65.2	45.70
W4A16KV8	-	-	50k	0.0	0.0	0.0	0.0	0.0	0.00
W4A16KV8	-	✓	50k	17.3	6.3	75.6	33.0	64.0	39.25
W4A16KV8	✓	✓	50k	23.3	15.0	79.6	33.7	57.0	41.72
BF16	-	-	1.2M	53.3	33.0	94.0	39.9	82.5	60.54
W4A16KV8	✓	✓	1.2M	46.6	36.6	89.6	37.8	80.0	58.12

adapters are trained on top of a frozen, quantized base model. To further improve memory and runtime efficiency, we quantize the trained LoRA adapter weights to INT8 and use INT16 activations during inference. As with the base model, we use symmetric per-channel quantization for weights and asymmetric per-tensor quantization for activations. We denote the aforementioned technique as *Quantization-Aware Modular Reasoning* (QAMR).

Training and evaluation details. We follow the training and evaluation protocols described in Section 3. For training, we use the OT3 [32] dataset. To assess the reasoning capabilities of our quantized reasoning model, we use a comprehensive set of benchmarks, including AIME 24/25 [35], MATH500 [37], GPQA Diamond [38], and AMC23 [36]. We conduct an ablation study to assess the contributions of both the improved base model quantization pipeline and the proposed QAMR approach using a random subset of 50k training examples, while reserving the full training dataset for the final set of results.

Results. We observe in Table 11 that a naïvely quantized base model combined with a full-precision reasoning module (i.e., without QARM) is essentially non-functional. Qualitatively such model outputs seemingly random tokens, without any structure or relevance to the tasks at hand.

In contrast, applying QARM – even with relatively short training – recovers a substantial portion of the performance on tasks such as MATH500 and GPQA. Notably, quantization-aware modular reasoning is *crucial for learning anything at all*.

Further, using FPTQuant^o offers a stronger starting point for training the reasoning module and consistently improves performance across all benchmarks except AMC23, for which longer training is required. By shaping the underlying activation distributions to be more quantization-friendly, using FPTQuant^o-quantized base model leads to significantly fewer training instabilities and enables faster learning compared to a base model quantized with a standard min-max range setting.

Finally, when combined with extended training, our approach achieves performance within roughly 2% of an equivalently trained full-precision reasoning model on average, while being significantly more compact, and inference-efficient.

7.4 Verifier Quantization and On-Device Deployment

As the final stage of our quantization pipeline, we address the quantization of the verifier, which is essential for deploying the proposed verifier on resource-constrained hardware. To minimize degradation from reduced numerical precision, we train the verifier directly on embeddings produced by the 4-bit weight-quantized Qwen-2.5-7B-Instruct model obtained in section 7.2. This choice reduces distribution shift between training and inference, ensuring that the verifier learns to operate on similar representations it will encounter at deployment time. After training

the verifier head under this setting, we further quantize both activations and verifier head weights to 8 bit representations using FastForward [18].

Once all components including the base model, reasoning adapters, and verifier are quantized, we prepare the models for on-device deployment. The first step after quantization is the model transformation to assure compatibility at pytorch representation level with the format supported by GENIE SDK [19]. These pertain to aspects of autoregressive parallel and sequential generation for prefill and decoding, as well as handling the attention operations and masking, and position embeddings.

Next, we establish compatibility with GENIE at the ONNX representation level. We use Qualcomm FastForward [18] for this stage and implement transformations at linear layers and multi-head attention, as well as model partitioning. We use Pytorch with FastForward to get the ONNX graph and the associated quantization encodings. We export to Deep Learning Container format, and Quantize any remaining non-quantized nodes missed by FastForward (e.g. biases). We compile for the deployment target (e.g. aarch64-android) and upload to the device using *adb* (Android Device Bridge).

8 Discussions and Challenges

Deploying capable reasoning models on resource-constrained edge devices requires navigating a complex trade-off between task performance, latency, memory footprint, and power consumption. In this work, we proposed a practical end-to-end framework to overcome these limitations. By decoupling reasoning from the base weights using modular LoRA adapters, we demonstrate that parameter-efficient fine-tuning can achieve competitive reasoning accuracy relative to computationally expensive full-parameter distillation methods like DeepSeek-R1-Distill. To optimize day-to-day user interactions, our lightweight dynamic Switcher routes standard conversations to the highly efficient base model, reserving the reasoning adapters strictly for complex queries where multi-step logic is required. To further combat latency, our budget-forced RL alignment explicitly penalizes generation verbosity, yielding a 2.4 $\times$ reduction in average reasoning tokens without sacrificing task accuracy. At inference time, we exploit the memory-bound nature of autoregressive decoding by introducing parallel test-time scaling, coupled with a lightweight verifier that provides up to a 10% accuracy boost on complex reasoning benchmarks. Finally, we show that 4-bit weight quantization via FPTQuant and Quantization-Aware Modular Reasoning preserves this robust performance, achieving within 2% of the full-precision reasoning model's accuracy while delivering massive memory savings. Below, we summarize the key insights derived from each component of our pipeline and outline the remaining challenges that pave the way for future research.

LoRA for modular reasoning. Our experiments revealed that parameter-efficient fine-tuning via LoRA is highly effective at eliciting reasoning capabilities in small (3B and 7B) base models, often rivaling the performance of computationally expensive full-parameter distillation. A key insight is that the success of LoRA is heavily dependent on adapter capacity and base model scale. While a LoRA rank of 128 allowed the 7B model to nearly match dense fine-tuning baselines on challenging benchmarks, the 3B model exhibited a wider performance gap, indicating that smaller backbones are more sensitive to adapter capacity limits. Furthermore, while reasoning specialization improves performance on complex tasks (e.g., LiveCodeBench), it introduces a trade-off, occasionally degrading zero-shot performance on simpler coding tasks that require direct answers. Managing this specialization-forgetting trade-off remains an ongoing challenge.

Dynamic LoRA routing via the switcher. We demonstrated that the computational overhead of reasoning can be drastically reduced by recognizing that not all queries require complex multi-step logic. The lightweight Switcher module successfully acts as an on-demand router, preserving the base model's speed for standard queries while activating reasoning LoRA adapters only when necessary. A major deployment insight was the necessity of *masked LoRA training* during the prefill stage. This strategy ensures that the KV-cache generated by the base model can be seamlessly reused by the reasoning adapters, completely eliminating the severe latency

penalty of re-encoding prompt tokens when switching modes.

While the current switcher relies on a supervised classifier head to evaluate query complexity, a promising direction for future work is to learn this routing policy via reinforcement learning. This would act synergistically with our budget-forcing objectives: while budget-forced RL explicitly shortens the reasoning traces when LoRA is active, an RL-driven router optimized for both accuracy and length would learn to bypass the adapters entirely whenever possible. Because the frozen base model natively produces direct answers without verbose chain-of-thought, successfully routing a query to the non-LoRA mode automatically yields a drastically shorter response, organically reserving the reasoning adapters strictly for complex queries where the base model would fail.

Future work could extend the switcher beyond binary routing and turn it into a general mechanism for dynamic LoRA selection. Instead of choosing only between the base model and a single reasoning adapter, the system could route each query to a bank of task-specific adapters [155; 156], for example specialized for mathematics, coding, or other domains, allowing the same backbone to support richer capabilities while preserving modular deployment. An especially promising direction is to include adapters tailored for latent reasoning, since recent work suggests that latent-reasoning LoRA adapters can retain strong reasoning performance while substantially reducing the token overhead of explicit CoT generation [157; 158; 159; 160; 161]. In this setting, the switcher would not only decide whether reasoning is needed, but also which form of reasoning is most efficient for a given query, making dynamic adapter routing a natural path toward more capable and more compute-efficient on-device systems.

Budget forcing. We presented our RL recipe to finetune LLMs to generate shorter completions. By leveraging a multiplicative penalty (eq. 3), we successfully aligned LLMs to reduce the number of generated tokens to answers given questions. Our empirical evaluations demonstrate an average completion length reduction of $2.4\times$ -and up to $8\times$ maximum compression-with minimal degradation in task accuracy. Crucially, our formulation structurally mitigates the reward hacking we observed in early experiments, ensuring that efficiency gains stem from genuine rationale compression rather than formatting exploits. While our “soft-barrier” approach offers a robust mechanism for compute-aware inference, it also opens several exciting avenues for future research. We outline these forward-looking directions, which naturally address the current boundaries of our methodology.

Our experiments provide a snapshot of budget forcing on state-of-the-art reasoning models. However, the relationship between base model scale (e.g., parameter count) and the capacity for rationale compression remains an open question. Investigating whether larger, more capable models exhibit greater “epistemic hesitation”, and thus offer a larger margin for ITC reduction, will be critical for formulating generalized scaling laws for budget forcing.

A fundamental limitation of current budget forcing approaches, including ours, is the assumption of uniform token cost where every generated token contributes equally to the budget consumption regardless of its utility. However, a token representing a crucial logical leap carries significantly higher semantic value than a token used for syntactic glue or hedging (e.g., “Let me think about this...”). A promising future direction lies in developing *semantic-aware* budget priors that weight penalties dynamically based on information density or local entropy as explored in [162]. By shifting the optimization objective from pure length minimization to *reasoning density maximization*, we can encourage models to prioritize high-utility tokens while disproportionately penalizing low-entropy filler, effectively decoupling computational cost from reasoning depth.

Parallel test-time scaling and reasoning. We have demonstrated that even a light verifier design can substantially improve the effectiveness of parallel test-time scaling for reasoning on edge devices. By combining the robustness of aggregation with a learned notion of correctness at negligible additional cost, weighted majority voting offers a practical and efficient approach for deploying parallel reasoning on resource-constrained devices. The current design can be extended to score the steps with a process reward model. Another interesting extension is the parallel reasoning schemes with interdependent generation as in [65; 67; 163].

Quantization Our experiments on Qwen2.5-7B-Instruct highlight the importance of starting from a strong quantized base model. Leveraging function-preserving transformations, improved range initialization, and joint fine-

tuning of transformation and quantization parameters offers a straightforward and lightweight path to quantizing modern LLMs while preserving robust predictive performance. When extending such models with reasoning capabilities, we further show that it is essential to account for distribution shifts induced by quantization. Inspired by prior PEFT literature, we address this challenge by proposing the Quantization-Aware Modular Reasoning (QAMR) approach, which mitigates the quantization noise and distribution shifts by training reasoning adapters directly on the quantized base model. Finally, our results demonstrate that a relatively compact 4-bit weight-quantized 7B model can achieve reasoning performance comparable to that of substantially larger models.

Reasoning remains a token-generation-intensive task, making it fundamentally memory-bound rather than compute-bound. As a result, further improvements in efficiency and performance will likely depend on reducing the memory footprint of the model weights. A promising direction for future work is to push quantization below 4 bits by leveraging state-of-the-art compression techniques such as Quip# [164], or exploring 2-3-bit QAT methods, such as ParetoQ [107].

9 Conclusion

In this work, we presented an end-to-end framework that makes state-of-the-art LLM reasoning practical on resource-constrained edge devices. We demonstrated that parameter-efficient LoRA adaptation, governed by a dynamic routing switcher, unlocks powerful reasoning capabilities without compromising the speed of everyday interactions. By introducing budget-forced reinforcement learning, we successfully curbed model verbosity to fit strict on-device token limits. To further maximize hardware utilization, we leveraged parallel test-time scaling and a lightweight latent verifier to boost accuracy during the memory-bound generation phase. Throughout this pipeline, hardware-awareness remains the unifying principle: from maximizing KV-cache reuse to intertwining quantization directly into the training of the adapters, switcher, and verifier. Ultimately, this co-designed approach bridges the gap between cloud-based reasoning and the strict memory, latency, and power budgets of mobile hardware, providing a practical blueprint for on-device AI.

References

- [1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. *Claude-3 Model Card*, 1(1):4, 2024. (Cited on page 1)
- [2] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024. (Cited on page 1, 3)
- [3] Mohammed Abouzaid, Andrew J Blumberg, Martin Hairer, Joe Kileel, Tamara G Kolda, Paul D Nelson, Daniel Spielman, Nikhil Srivastava, Rachel Ward, Shmuel Weinberger, and Lauren Williams. First proof. *arXiv preprint arXiv:2602.05192*, 5 February 2026. (Cited on page 1)
- [4] Yang Cao, Yubin Chen, Xuyang Guo, Zhao Song, Song Yue, Jiahao Zhang, and Jiale Zhao. Evaluating frontier LLMs on PhD-level mathematical reasoning: A benchmark on a textbook in theoretical computer science about randomized algorithms. *arXiv [cs.AI]*, 16 December 2025. (Cited on page 1)
- [5] Tony Feng, Trieu H Trinh, Garrett Bingham, Dawsen Hwang, Yuri Chervonyi, Junehyuk Jung, Joonkyung Lee, Carlo Pagano, Sang-Hyun Kim, Federico Pasqualotto, Sergei Gukov, Jonathan N Lee, Junsu Kim, Kaiying Hou, Golnaz Ghiasi, Yi Tay, Yaguang Li, Chenkai Kuang, Yuan Liu, Hanzhao Lin, Evan Zheran Liu, Nigamaa Nayakanti, Xiaomeng Yang, Heng-Tze Cheng, Demis Hassabis, Koray Kavukcuoglu, Quoc V Le, and Thang Luong. Towards autonomous mathematics research. *arXiv preprint arXiv:2602.10177*, 12 February 2026. (Cited on page 1)
- [6] David P Woodruff, Vincent Cohen-Addad, Lalit Jain, Jieming Mao, Song Zuo, Mohammadhossein Bateni, Simina Branzei, Michael P Brenner, Lin Chen, Ying Feng, Lance Fortnow, Gang Fu, Ziyi Guan, Zahra Hadizadeh, Mohammad T Hajiaghayi, Mahdi JafariRaviz, Adel Javanmard, Karthik C. S., Ken-Ichi Kawarabayashi, Ravi Kumar, Silvio Lattanzi, Euiwoong Lee, Yi Li, Ioannis Panageas, Dimitris Paparas, Benjamin Przybocki, Bernardo Subercaseaux, Ola Svensson, Shayan Taherijam, Xuan Wu, Eylon Yogev, Morteza Zadimoghaddam, Samson Zhou, Yossi Matias, James Manyika, and Vahab Mirrokni. Accelerating scientific research with gemini: Case studies and common techniques. *arXiv preprint arXiv:2602.03837*, 16 February 2026. (Cited on page 1)
- [7] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 29 September 2022. (Cited on page 1)
- [8] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. MAI-UI technical report: Real-world centric foundation GUI agents. *arXiv [cs.CV]*, 26 December 2025. (Cited on page 1)
- [9] Haoming Wang, Haoyang Zou, Huatong Song, Jiazhan Feng, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, Wanjun Zhong, Yining Ye, Yujia Qin, Yuwen Xiong, Yuxin Song, Zhiyong Wu, Aoyan Li, Bo Li, Chen Dun, Chong Liu, Daoguang Zan, Fuxing Leng, Hanbin Wang, Hao Yu, Haobin Chen, Hongyi Guo, Jing Su, Jingjia Huang, Kai Shen, Kaiyu Shi, Lin Yan, Peiyao Zhao, Pengfei Liu, Qinghao Ye, Renjie Zheng, Shulin Xin, Wayne Xin Zhao, Wen Heng, Wenhao Huang, Wenqian Wang, Xiaobo Qin, Yi Lin, Youbin Wu, Zehui Chen, Zihao Wang, Baoquan Zhong, Xinchun Zhang, Xujing Li, Yuanfan Li, Zhongkai Zhao, Chengquan Jiang, Faming Wu, Haotian Zhou, Jinlin Pang, Li Han, Qi Liu, Qianli Ma, Siyao Liu, Songhua Cai, Wenqi Fu, Xin Liu, Yaohui Wang, Zhi Zhang, Bo Zhou, Guoliang Li, Jiajun Shi, Jiale Yang, Jie Tang, Li Li, Qihua Han, Taoran Lu, Woyu Lin, Xiaokang Tong, Xinyao Li, Yichi Zhang, Yu Miao, Zhengxuan Jiang, Zili Li, Ziyuan Zhao, Chenxin Li, Dehua Ma, Feng Lin, Ge Zhang, Haihua Yang, Hangyu Guo, Hongda Zhu, Jiaheng Liu, Junda Du, Kai Cai, Kuanye Li, Lichen Yuan, Meilan Han, Minchao Wang, Shuyue Guo, Tianhao Cheng, Xiaobo Ma, Xiaojun Xiao, Xiaolong Huang, Xinjie Chen, Yidi Du, Yilin Chen, Yiwen Wang, Zhaojian Li, Zhenzhu Yang, Zhiyuan Zeng,

Chaolin Jin, Chen Li, Hao Chen, Haoli Chen, Jian Chen, Qinghao Zhao, and Guang Shi. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv [cs.AI]*, 5 September 2025. (Cited on page 1)

- [10] Venus Team, Changlong Gao, Zhangxuan Gu, Yulin Liu, Xinyu Qiu, Shuheng Shen, Yue Wen, Tianyu Xia, Zhenyu Xu, Zhengwen Zeng, et al. UI-venus-1.5 technical report. *arXiv [cs.CV]*, 9 February 2026. (Cited on page 1)
- [11] Measuring thinking efficiency in reasoning models: The missing benchmark. <https://nousresearch.com/measuring-thinking-efficiency-in-reasoning-models-the-missing-benchmark/>, 14 August 2025. Accessed: 2026-2-19. (Cited on page 1)
- [12] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, S Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *ACL*, 2024. (Cited on page 2)
- [13] Jie Xiao, Qianyi Huang, Xu Chen, and Chen Tian. Understanding large language models in your pockets: Performance study on COTS mobile devices. *arXiv [cs.LG]*, 9 February 2026. (Cited on page 2)
- [14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. (Cited on page 2, 4)
- [15] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, and et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>. (Cited on page 2, 4)
- [16] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025. doi: 10.48550/arXiv.2501.19393. URL <https://arxiv.org/abs/2501.19393>. (Cited on page 2, 10, 11)
- [17] Junyan Li, Wenshuo Zhao, Yang Zhang, and Chuang Gan. Steering llm thinking with budget guidance. *arXiv preprint arXiv:2506.13752*, 2025. (Cited on page 2)
- [18] Fastforward: Neural network quantization for research and prototyping. <https://github.com/Qualcomm-AI-research/fastforward>. (Cited on page 3, 20, 23)
- [19] Qualcomm gen ai inference extensions (genie). <https://www.qualcomm.com/developer/software/gen-ai-inference-extensions>. (Cited on page 3, 23)
- [20] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022. (Cited on page 3)
- [21] Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2022. URL <https://openreview.net/forum?id=iedYJm92o0a>. (Cited on page 3)
- [22] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf. (Cited on page 3, 11)

-
- [23] Shangshang Wang, Julian Asilis, Ömer Faruk Akgül, Enes Burak Bilgin, Ollie Liu, and Willie Neiswanger. Tina: Tiny reasoning models via lora. *arXiv preprint arXiv:2504.15777*, 2025. (Cited on page 3)
- [24] Haoran Xu, Baolin Peng, Hany Awadalla, Dongdong Chen, Yen-Chun Chen, Mei Gao, Young Jin Kim, Yunsheng Li, Liliang Ren, Yelong Shen, et al. Phi-4-mini-reasoning: Exploring the limits of small reasoning language models in math. *arXiv preprint arXiv:2504.21233*, 2025. (Cited on page 3)
- [25] Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, Li Dong, Xingxing Zhang, Tengchao Lv, Lei Cui, and Furu Wei. Think only when you need with large hybrid-reasoning models. *arXiv preprint arXiv:2505.14631*, 2025. (Cited on page 3)
- [26] John Schulman and Thinking Machines Lab. Lora without regret, 2025. <https://thinkingmachines.ai/blog/lora/>. (Cited on page 4)
- [27] Yang Chen, Zhuolin Yang, Zihan Liu, Chankyu Lee, Peng Xu, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Acereason-nemotron: Advancing math and code reasoning through reinforcement learning. *arXiv preprint arXiv:2505.16400*, 2025. (Cited on page 4)
- [28] Quy-Anh Dang and Chris Ngo. Reinforcement learning for reasoning in small llms: What works and what doesn't. *arXiv preprint arXiv:2503.16219*, 2025. (Cited on page 4)
- [29] Mohammad Ali Alomrani, Yingxue Zhang, Derek Li, Qianyi Sun, Soumyasundar Pal, Zhanguang Zhang, Yaochen Hu, Rohan Deepak Ajwani, Antonios Valkanas, Raika Karimi, et al. Reasoning on a budget: A survey of adaptive and controllable test-time compute in llms. *arXiv preprint arXiv:2507.02076*, 2025. (Cited on page 4)
- [30] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. (Cited on page 4, 12)
- [31] Qwen Team. Qwq-32b: Embracing the power of reinforcement learning. <https://qwen.ai/blog?id=qwq-32b>, 2025. (Cited on page 4)
- [32] Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025. (Cited on page 4, 5, 7, 22)
- [33] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. URL <https://arxiv.org/abs/2412.15115>. (Cited on page 5)
- [34] Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL <https://github.com/huggingface/open-r1>. (Cited on page 5)
- [35] Art of problem solving. aime problems and solutions. 2024. URL https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions. (Cited on page 5, 22, 39)
- [36] Art of problem solving. amc problems and solutions. 2023. URL https://artofproblemsolving.com/wiki/index.php/AMC_12_Problems_and_Solutions. (Cited on page 5, 22, 39)
- [37] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021. (Cited on page 5, 10, 17, 22, 39)

-
- [38] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. (Cited on page 5, 22, 39)
- [39] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv [cs.SE]*, March 2024. (Cited on page 5, 39)
- [40] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv [cs.LG]*, July 2021. (Cited on page 5, 6, 39)
- [41] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv [cs.PL]*, August 2021. (Cited on page 5, 39)
- [42] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. *arXiv [cs.SE]*, May 2023. (Cited on page 5, 39)
- [43] Nathan Habib, Clémentine Fourrier, Hynek Kydliček, Thomas Wolf, and Lewis Tunstall. Lighteval: A lightweight framework for llm evaluation, 2023. URL <https://github.com/huggingface/lighteval>. (Cited on page 6)
- [44] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. (Cited on page 6)
- [45] Maggie Huan, Yuetai Li, Tuney Zheng, Xiaoyu Xu, Seungone Kim, Minxin Du, Radha Poovendran, Graham Neubig, and Xiang Yue. Does math reasoning improve general llm capabilities? understanding transferability of llm reasoning. *arXiv preprint arXiv:2507.00432*, 2025. (Cited on page 7)
- [46] Song Lai, Haohan Zhao, Rong Feng, Changyi Ma, Wenzhuo Liu, Hongbo Zhao, Xi Lin, Dong Yi, Qingfu Zhang, Hongbin Liu, et al. Reinforcement fine-tuning naturally mitigates forgetting in continual post-training. *arXiv preprint arXiv:2507.05386*, 2025. (Cited on page 7)
- [47] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for squad, 2018. URL <https://arxiv.org/abs/1806.03822>. (Cited on page 10)
- [48] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies, 2021. URL <https://arxiv.org/abs/2101.02235>. (Cited on page 10)
- [49] Junyu Zhang, Yifan Sun, Tianang Leng, Jingyan Shen, Liu Ziyin, Paul Pu Liang, and Huan Zhang. When reasoning meets its laws. In *NeurIPS 2025 Workshop on Efficient Reasoning*, 2025. URL <https://openreview.net/forum?id=lWjcbodr4M>. (Cited on page 11)

-
- [50] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025. URL <https://arxiv.org/pdf/2502.18600>. (Cited on page 11)
- [51] Matthew Renze and Erhan Guven. The benefits of a concise chain of thought on problem-solving in large language models. *arXiv preprint arXiv:2401.05618*, 2024. URL <https://arxiv.org/pdf/2401.05618v1.pdf>. (Cited on page 11)
- [52] Jiaqi Wang, Kevin Qinghong Lin, James Cheng, and Mike Zheng Shou. Think or not? selective reasoning via reinforcement learning for vision-language models. *arXiv preprint arXiv:2505.16854*, 2025. URL <https://arxiv.org/pdf/2505.16854>. (Cited on page 11)
- [53] Chengyu Huang, Zhengxin Zhang, and Claire Cardie. Hapo: Training language models to reason concisely via history-aware policy optimization. *arXiv preprint arXiv:2505.11225*, 2025. (Cited on page 11)
- [54] Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025. URL <https://arxiv.org/pdf/2503.04697>. (Cited on page 11)
- [55] Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, et al. Dler: Doing length penalty right-incentivizing more intelligence per token via reinforcement learning. *arXiv preprint arXiv:2510.15110*, 2025. (Cited on page 11)
- [56] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Li Erran Li, et al. Deepscaler: Surpassing o1-preview with a 1.5 b model by scaling rl. *Notion Blog*, 2025. (Cited on page 12)
- [57] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL <https://github.com/huggingface/trl>. (Cited on page 13)
- [58] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, volume 2024, pages 39578–39601, 2024. (Cited on page 13, 16)
- [59] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. (Cited on page 16)
- [60] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. (Cited on page 16)
- [61] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024. (Cited on page 16)
- [62] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for llm problem-solving. In *The Thirteenth International Conference on Learning Representations*, 2025. (Cited on page 16, 17)
- [63] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. In *The Thirteenth International Conference on Learning Representations*, 2025. (Cited on page 16)

-
- [64] Matthew Renze. The effect of sampling temperature on problem solving in large language models. In *Findings of the association for computational linguistics: EMNLP 2024*, pages 7346–7356, 2024. (Cited on page 16)
- [65] Chan-Jan Hsu, Davide Buffelli, Jamie McGowan, Feng-Ting Liao, Yi-Chang Chen, Sattar Vakili, and Da-shan Shiu. Group think: Multiple concurrent reasoning agents collaborating at token level granularity. *arXiv preprint arXiv:2505.11107*, 2025. (Cited on page 16, 24)
- [66] Jiayi Pan, Xiuyu Li, Long Lian, Charlie Snell, Yifei Zhou, Adam Yala, Trevor Darrell, Kurt Keutzer, and Alane Suhr. Learning adaptive parallel reasoning with language models. *arXiv preprint arXiv:2504.15466*, 2025. (Cited on page 16)
- [67] Gleb Rodionov, Roman Garipov, Alina Shutova, George Yakushev, Erik Schultheis, Vage Egiazarian, Anton Sinitsin, Denis Kuznedelev, and Dan Alistarh. Hogwild! inference: Parallel llm generation via concurrent attention. *arXiv preprint arXiv:2504.06261*, 2025. (Cited on page 16, 24)
- [68] Tong Zheng, Hongming Zhang, Wenhao Yu, Xiaoyang Wang, Xinyu Yang, Runpeng Dai, Rui Liu, Huiwen Bao, Chengsong Huang, Heng Huang, et al. Parallel-r1: Towards parallel thinking via reinforcement learning. *arXiv preprint arXiv:2509.07980*, 2025. (Cited on page 16)
- [69] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023. (Cited on page 16)
- [70] Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. Skeleton-of-thought: Prompting llms for efficient parallel generation. *arXiv preprint arXiv:2307.15337*, 2023. (Cited on page 16)
- [71] Shuangtao Li, Shuaihao Dong, Kexin Luan, Xinhan Di, and Chaofan Ding. Enhancing reasoning through process supervision with monte carlo tree search. *arXiv preprint arXiv:2501.01478*, 2025. (Cited on page 16)
- [72] Chris Yuhao Liu, Liang Zeng, Jiakai Liu, Rui Yan, Jujie He, Chaojie Wang, Shuicheng Yan, Yang Liu, and Yahui Zhou. Skywork-reward: Bag of tricks for reward modeling in llms. *arXiv preprint arXiv:2410.18451*, 2024. (Cited on page 16)
- [73] Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J Zhang, Makesh N Sreedhar, and Oleksii Kuchaiev. Helpsteer 2: Open-source dataset for training top-performing reward models, 2024. (Cited on page 16)
- [74] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *arXiv preprint arXiv:2312.08935*, 2023. (Cited on page 16)
- [75] Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025. (Cited on page 16)
- [76] Zijun Liu, Peiyi Wang, Runxin Xu, Shirong Ma, Chong Ruan, Peng Li, Yang Liu, and Yu Wu. Inference-time scaling for generalist reward modeling. *arXiv preprint arXiv:2504.02495*, 2025. (Cited on page 16)
- [77] Hyunjoo Chae, Sunghwan Kim, Junhee Cho, Seungone Kim, Seungjun Moon, Gyeom Hwangbo, Dongha Lim, Minjin Kim, Yeonjun Hwang, Minju Gwak, et al. Web-shepherd: Advancing prms for reinforcing web agents. *arXiv preprint arXiv:2505.15277*, 2025. (Cited on page 16)

-
- [78] Zhenfang Chen, Delin Chen, Rui Sun, Wenjun Liu, and Chuang Gan. Scaling autonomous agents via automatic reward modeling and planning. *arXiv preprint arXiv:2502.12130*, 2025. (Cited on page 16)
- [79] Ammar Khairi, Daniel D'souza, Marzieh Fadaee, and Julia Kreutzer. Making, not taking, the best of n. *arXiv preprint arXiv:2510.00931*, 2025. (Cited on page 17)
- [80] Yafu Li, Zhilin Wang, Tingchen Fu, Ganqu Cui, Sen Yang, and Yu Cheng. From drafts to answers: Unlocking llm potential via aggregation fine-tuning. *arXiv preprint arXiv:2501.11877*, 2025. (Cited on page 17)
- [81] Jianing Qi, Xi Ye, Hao Tang, Zhigang Zhu, and Eunsol Choi. Learning to reason across parallel samples for llm reasoning. *arXiv preprint arXiv:2506.09014*, 2025. (Cited on page 17)
- [82] Wenting Zhao, Pranjal Aggarwal, Swarnadeep Saha, Asli Celikyilmaz, Jason Weston, and Ilia Kulikov. The majority is not always right: RL training for solution aggregation. *arXiv preprint arXiv:2509.06870*, 2025. (Cited on page 17)
- [83] Zixu Hao, Jianyu Wei, Tuowei Wang, Minking Huang, Huiqiang Jiang, Shiqi Jiang, Ting Cao, and Ju Ren. Scaling llm test-time compute with mobile npu on smartphones. In *EuroSys 2026*. ACM, November 2026. URL <https://www.microsoft.com/en-us/research/publication/scaling-llm-test-time-compute-with-mobile-npu-on-smartphones/>. (Cited on page 17)
- [84] Hanshi Sun, Momin Haider, Ruiqi Zhang, Huitao Yang, Jiahao Qiu, Ming Yin, Mengdi Wang, Peter Bartlett, and Andrea Zanette. Fast best-of-n decoding via speculative rejection. *Advances in Neural Information Processing Systems*, 37:32630–32652, 2024. (Cited on page 17)
- [85] Coleman Hooper, Sehoon Kim, Suhong Moon, Kerem Dilmen, Monishwaran Maheswaran, Nicholas Lee, Michael W Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. Ets: Efficient tree search for inference-time scaling. *arXiv preprint arXiv:2502.13575*, 2025. (Cited on page 17)
- [86] Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. Generative verifiers: Reward modeling as next-token prediction. In *The Thirteenth International Conference on Learning Representations*, 2025. (Cited on page 17)
- [87] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv preprint arXiv:1806.08342*, 2018. (Cited on page 19)
- [88] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021. (Cited on page 19)
- [89] M. Horowitz. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014. doi: 10.1109/ISSCC.2014.6757323. (Cited on page 19)
- [90] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *The Journal of Machine Learning Research*, 18(1):6869–6898, 2017. (Cited on page 19)
- [91] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016. (Cited on page 19)
- [92] Ron Banner, Yury Nahshan, Elad Hoffer, and Daniel Soudry. Post-training 4-bit quantization of convolution networks for rapid-deployment. *arXiv preprint arXiv:1810.05723*, 2018. (Cited on page 19)

-
- [93] Yaohui Cai, Zhewei Yao, Zhen Dong, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. Zeroq: A novel zero shot quantization framework. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13169–13178, 2020. (Cited on page 19)
- [94] Yoni Choukroun, Eli Kravchik, Fan Yang, and Pavel Kisilev. Low-bit quantization of neural networks for efficient inference. In *ICCV Workshops*, pages 3009–3018, 2019. (Cited on page 19)
- [95] Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. Improving post training neural quantization: Layer-wise calibration and integer programming. *arXiv preprint arXiv:2006.10518*, 2020. (Cited on page 19)
- [96] Eldad Meller, Alexander Finkelstein, Uri Almog, and Mark Grobman. Same, same but different: Recovering neural network quantization error through weight factorization. In *International Conference on Machine Learning*, pages 4486–4495. PMLR, 2019. (Cited on page 19)
- [97] Ritchie Zhao, Yuwei Hu, Jordan Dotzel, Chris De Sa, and Zhiru Zhang. Improving neural network quantization without retraining using outlier channel splitting. In *International conference on machine learning*, pages 7543–7552. PMLR, 2019. (Cited on page 19)
- [98] Markus Nagel, Mart van Baalen, Tijmen Blankevoort, and Max Welling. Data-free quantization through weight equalization and bias correction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019. (Cited on page 19, 20)
- [99] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or Down? Adaptive Rounding for Post-Training Quantization, April 2020. URL <https://arxiv.org/abs/2004.10568v2>. (Cited on page 19)
- [100] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. Brecq: Pushing the limit of post-training quantization by block reconstruction. *arXiv preprint arXiv:2102.05426*, 2021. (Cited on page 19)
- [101] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. In *International conference on machine learning*, pages 1737–1746. PMLR, 2015. (Cited on page 19)
- [102] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. (Cited on page 19)
- [103] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. Learned step size quantization. In *International Conference on Learning Representations (ICLR)*, 2020. (Cited on page 19)
- [104] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *International Conference on Machine Learning*, pages 16318–16330. PMLR, 2022. (Cited on page 19)
- [105] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. Llm-qat: Data-free quantization aware training for large language models. *arXiv preprint arXiv:2305.17888*, 2023. (Cited on page 19)
- [106] Dayou Du, Yijia Zhang, Shijie Cao, Jiaqi Guo, Ting Cao, Xiaowen Chu, and Ningyi Xu. Bitdistiller: Unleashing the potential of sub-4-bit llms via self-distillation. *arXiv preprint arXiv:2402.10631*, 2024. (Cited on page 19)

-
- [107] Zechun Liu, Changsheng Zhao, Hanxian Huang, Sijia Chen, Jing Zhang, Jiawei Zhao, Scott Roy, Lisa Jin, Yunyang Xiong, Yangyang Shi, Lin Xiao, Yuandong Tian, Bilge Soran, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. Paretoq: Improving scaling laws in extremely low-bit llm quantization, 2025. URL <https://arxiv.org/abs/2502.02631>. (Cited on page 19, 25)
- [108] Mengzhao Chen, Chaoyi Zhang, Jing Liu, Yutao Zeng, Zeyue Xue, Zhiheng Liu, Yunshui Li, Jin Ma, Jie Huang, Xun Zhou, et al. Scaling law for quantization-aware training. *arXiv preprint arXiv:2505.14302*, 2025. (Cited on page 19)
- [109] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024. (Cited on page 19, 21)
- [110] Yuhui Xu, Lingxi Xie, Xiaotao Gu, Xin Chen, Heng Chang, Hengheng Zhang, Zhengsu Chen, Xiaopeng Zhang, and Qi Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023. (Cited on page 19, 21)
- [111] Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. Loftq: Lora-fine-tuning-aware quantization for large language models. *arXiv preprint arXiv:2310.08659*, 2023. (Cited on page 19, 21)
- [112] Han Guo, Philip Greengard, Eric P Xing, and Yoon Kim. Lq-lora: Low-rank plus quantized matrix decomposition for efficient language model finetuning. *arXiv preprint arXiv:2311.12023*, 2023. (Cited on page 19)
- [113] Jeonghoon Kim, Jung Hyun Lee, Sungdong Kim, Joonsuk Park, Kang Min Yoo, Se Jung Kwon, and Dongsoo Lee. Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization. *Advances in Neural Information Processing Systems*, 36, 2024. (Cited on page 19)
- [114] Yelysei Bondarenko, Riccardo Del Chiaro, and Markus Nagel. Low-rank quantization-aware training for llms. *arXiv preprint arXiv:2406.06385*, 2024. (Cited on page 19)
- [115] Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Understanding and overcoming the challenges of efficient transformer quantization. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7947–7969, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.627. URL <https://aclanthology.org/2021.emnlp-main.627>. (Cited on page 19)
- [116] Olga Kovaleva, Saurabh Kulshreshtha, Anna Rogers, and Anna Rumshisky. Bert busters: Outlier dimensions that disrupt transformers. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3392–3405, 2021. (Cited on page 19)
- [117] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, 2022. (Cited on page 19)
- [118] Yelysei Bondarenko, Markus Nagel, and Tijmen Blankevoort. Quantizable Transformers: Removing Outliers by Helping Attention Heads Do Nothing. *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.12929v2>. (Cited on page 19)
- [119] Mingjie Sun, Xinlei Chen, J Zico Kolter, and Zhuang Liu. Massive activations in large language models. *arXiv preprint arXiv:2402.17762*, 2024. (Cited on page 19)
- [120] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022. (Cited on page 19, 20)
- [121] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*, 2023. (Cited on page 19)

-
- [122] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023. (Cited on page 19)
- [123] Changhun Lee, Jungyu Jin, Taesu Kim, Hyungjun Kim, and Eunhyeok Park. Owq: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 13355–13364, 2024. (Cited on page 19)
- [124] Yongkweon Jeon, Chungman Lee, Kyungphil Park, and Ho-young Kim. A frustratingly easy post-training quantization scheme for llms. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14446–14461, 2023. (Cited on page 19)
- [125] Jung Hyun Lee, Jeonghoon Kim, Se Jung Kwon, and Dongsoo Lee. Flexround: Learnable rounding based on element-wise division for post-training quantization. In *International Conference on Machine Learning*, pages 18913–18939. PMLR, 2023. (Cited on page 19)
- [126] Yan Luo, Yangcheng Gao, Zhao Zhang, Jicong Fan, Haijun Zhang, and Mingliang Xu. Long-range zero-shot generative deep network quantization. *Neural Networks*, 166:683–691, 2023. (Cited on page 19)
- [127] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36, 2024. (Cited on page 19, 20)
- [128] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR, 2023. (Cited on page 19, 20)
- [129] Xiuying Wei, Yunchen Zhang, Xiangguo Zhang, Ruihao Gong, Shanghang Zhang, Qi Zhang, Fengwei Yu, and Xianglong Liu. Outlier suppression: Pushing the limit of low-bit transformer language models. *arXiv preprint arXiv:2209.13325*, 2022. (Cited on page 19)
- [130] Janghwan Lee, Minsoo Kim, Seungcheol Baek, Seok Joong Hwang, Wonyong Sung, and Jungwook Choi. Enhancing computation efficiency in large language models through weight and activation quantization. *arXiv preprint arXiv:2311.05161*, 2023. (Cited on page 19)
- [131] Jing Liu, Ruihao Gong, Xiuying Wei, Zhiwei Dong, Jianfei Cai, and Bohan Zhuang. Qllm: Accurate and efficient low-bitwidth quantization for large language models. *arXiv preprint arXiv:2310.08041*, 2023. (Cited on page 19)
- [132] Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo Zhang, Ruihao Gong, Jinyang Guo, and Xianglong Liu. Outlier Suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling, October 2023. URL <http://arxiv.org/abs/2304.09145>. arXiv:2304.09145 [cs]. (Cited on page 19, 20)
- [133] Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xinggang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. Rptq: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089*, 2023. (Cited on page 19)
- [134] Hanlin Tang, Yifu Sun, Decheng Wu, Kai Liu, Jianchen Zhu, and Zhanhui Kang. Easyquant: An efficient data-free quantization algorithm for llms. *arXiv preprint arXiv:2403.02775*, 2024. (Cited on page 19)
- [135] Zhewei Yao, Reza Yazdani Aminabadi, Minjia Zhang, Xiaoxia Wu, Conglong Li, and Yuxiong He. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances*

-
- in *Neural Information Processing Systems*, volume 35, pages 27168–27183. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/adf7fa39d65e2983d724ff7da57f00ac-Paper-Conference.pdf. (Cited on page 19)
- [136] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532*, 2024. (Cited on page 19)
- [137] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models, March 2024. URL <http://arxiv.org/abs/2308.13137>. arXiv:2308.13137 [cs]. (Cited on page 20)
- [138] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs, March 2024. URL <https://arxiv.org/abs/2404.00456v1>. (Cited on page 20)
- [139] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. SpinQuant: LLM quantization with learned rotations, May 2024. URL <https://arxiv.org/abs/2405.16406v2>. (Cited on page 20)
- [140] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. DuQuant: Distributing Outliers via Dual Transformation Makes Stronger Quantized LLMs. In *Advances in Neural Information Processing Systems*. arXiv, November 2024. doi: 10.48550/arXiv.2406.01721. URL <http://arxiv.org/abs/2406.01721>. arXiv:2406.01721. (Cited on page 20)
- [141] Xing Hu, Yuan Cheng, Dawei Yang, Zukang Xu, Zhihang Yuan, Jiangyong Yu, Chen Xu, Zhe Jiang, and Sifan Zhou. OSTQuant: Refining Large Language Model Quantization with Orthogonal and Scaling Transformations for Better Distribution Fitting. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=rAcgDBdKnP>. (Cited on page 20)
- [142] Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, Wulong Liu, and Jun Yao. Flatquant: Flatness matters for LLM quantization. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=uTz2Uty5n>. (Cited on page 20)
- [143] Boris van Breugel, Yelysei Bondarenko, Paul Whatmough, and Markus Nagel. Fptquant: Function-preserving transforms for llm quantization. *arXiv preprint arXiv:2506.04985*, 2025. (Cited on page 20)
- [144] Jung Hyun Lee, Seungjae Shin, Vinnam Kim, Jaeseong You, and An Chen. Unifying block-wise ptq and distillation-based qat for progressive quantization toward 2-bit instruction-tuned llms. *arXiv preprint arXiv:2506.09104*, 2025. (Cited on page 20)
- [145] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. SmolLM2: When smol goes big—data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025. (Cited on page 20)
- [146] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Guha, Sedrick Scott Keh, Kushal Arora, et al. DataComp-1M: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37:14200–14282, 2024. (Cited on page 20)
- [147] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin A Raffel, Leandro Von Werra, Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in Neural Information Processing Systems*, 37:30811–30849, 2024. (Cited on page 20)

-
- [148] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about Physical Commonsense in Natural Language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05): 7432–7439, April 2020. ISSN 2374-3468. doi: 10.1609/aaai.v34i05.6239. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>. Number: 05. (Cited on page 20)
- [149] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64(9):99–106, August 2021. ISSN 0001-0782. doi: 10.1145/3474381. URL <https://dl.acm.org/doi/10.1145/3474381>. (Cited on page 20)
- [150] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a Machine Really Finish Your Sentence?, May 2019. URL <http://arxiv.org/abs/1905.07830>. arXiv:1905.07830 [cs]. (Cited on page 20)
- [151] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge, March 2018. URL <http://arxiv.org/abs/1803.05457>. arXiv:1803.05457 [cs]. (Cited on page 20)
- [152] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, 2016. (Cited on page 20)
- [153] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020. (Cited on page 20)
- [154] Lintang Sutawika, Hailey Schoelkopf, Leo Gao, Baber Abbasi, Stella Biderman, Jonathan Tow, Charles Lovering, Jason Phang, Anish Thite, Thomas Wang, et al. Eleutherai/lm-evaluation-harness: v0.4.9. *Zenodo*, 2025. (Cited on page 20)
- [155] Shaohan Huang and Furu Wei. Mixture of lora experts. In *ICLR 2024*, April 2024. (Cited on page 24)
- [156] Wenfeng Feng, Chuzhan Hao, Yuewei Zhang, Yu Han, and Hao Wang. Mixture-of-loras: An efficient multi-task tuning method for large language models. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 11371–11380, 2024. (Cited on page 24)
- [157] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2024. URL <https://arxiv.org/abs/2412.06769>. (Cited on page 24)
- [158] Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. Codi: Compressing chain-of-thought into continuous space via self-distillation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 677–693, 2025. (Cited on page 24)
- [159] Haoyi Wu, Zhihao Teng, and Kewei Tu. Parallel continuous chain-of-thought with jacobi iteration. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 914–926, 2025. (Cited on page 24)
- [160] Anna Kuzina, Maciej Pióro, Paul N. Whatmough, and Babak Ehteshami Bejnordi. Kava: Latent reasoning via compressed KV-cache distillation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=ePrhcLbtGv>. (Cited on page 24)
- [161] Xiaoqiang Wang, Suyuchen Wang, Yun Zhu, and Bang Liu. System-1.5 reasoning: Traversal in language and latent spaces with dynamic shortcuts. *arXiv preprint arXiv:2505.18962*, 2025. (Cited on page 24)

-
- [162] Fabio Valerio Massoli, Andrey Kuzmin, and Arash Behboodi. Reasoning as compression: Unifying budget forcing via the conditional information bottleneck. *arXiv preprint arXiv:2603.08462*, 9 March 2026. (Cited on page 24)
- [163] Gabriele Cesa, Thomas Hehn, Aleix Torres-Camps, Àlex Batlle Casellas, Jordi Ros-Giralt, Arash Behboodi, and Tribhuvanesh Orekondy. LaneroPE: Positional encoding for collaborative parallel reasoning and generation. In *Workshop on Multi-Agent Learning and Its Opportunities in the Era of Generative AI*, 2026. URL <https://openreview.net/forum?id=6WAuvwZjmw>. (Cited on page 24)
- [164] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. Quip#: Even better llm quantization with hadamard incoherence and lattice codebooks. *Proceedings of machine learning research*, 235:48630, 2024. (Cited on page 25)

A Benchmark Description

To comprehensively assess the reasoning capabilities of our fine-tuned models, we leverage a diverse suite of benchmarks spanning the mathematics, science, and coding domains.

- **AIME 24/25** [35] consists of 30 highly challenging mathematics competition problems from the 2024 and 2025 American Invitational Mathematics Examination. The questions cover a range of topics including algebra, geometry, and number theory, and are aimed for high-school students. The problems require multi-step reasoning, and the answers are integers between 0 and 999.
- **MATH500** [37] is a benchmark consisting of 500 mathematical questions spanning different topics including algebra, geometry, number theory, precalculus, and probability. The problems require multi-step solutions, and answers may include LaTeX-formatted expressions.
- **GPQA Diamond** [38] consists of 198 science PhD-level questions from physics, chemistry, and biology. All questions are presented in a multiple-choice format.
- **AMC23** [36] contains 40 problems from the 2023 American Mathematics Competition with integer answers.
- **LiveCodeBench** [39] is a continuously updated (hence "live") coding benchmark. In every release, a certain number of coding problems is sourced from competitive programming platforms (e.g. CodeForces, LeetCode), and each problem is used to build four coding "scenarios": Code Generation, Code Repair, Test Output Prediction, and Code Execution. In this work we use the v2 release, comprising 511 problems, and confine ourselves to the Code Generation scenario.
- **HumanEval and HumanEval+** [40; 42] are the original and improved versions of the HumanEval benchmark, comprising 164 problems in which a model, based on a Python function's signature and docstring, must generate its body. The resulting function is then verified with several unit tests. **HumanEval+** [42] improves the original benchmark by increasing the number unit tests for verification by 80 times.
- **MBPP and MBPP+** [41; 42] are the original and improved versions of the Most Basic Python Programs benchmark, consisting of 1000 basic Python programming tasks sourced from human coders. Each task involves writing a simple Python function based on natural language requirements and three unit tests it must pass. The **MBPP+** enhancement selects a subset of 378 task and increases the number of unit tests by 35 times.

B LoRA ablation study

This appendix provides the complete, detailed results of the parameter-efficient fine-tuning (PEFT) ablation study introduced in Section 3.4.1, in Tables 12 and 13. The study explores the impact of varying learning rates, batch sizes, and LoRA adapter ranks on the reasoning capabilities of both the 3B and 7B model backbones. The models were trained on a 50,000-entry subset of the OpenThoughts3 (OT3) dataset for one epoch and evaluated across core mathematical and scientific benchmarks (AIME24, AIME25, MATH500, GPQA, and AMC23).

Table 12: Ablation results for Qwen2.5-3B-Instruct. LR denotes learning rate and BS stands for batch size. In each learning rate subgroup, the best performance is marked in bold.

LR	BS	Rank	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
0.0001	32	32	0.05	0.01	0.55	0.25	0.24	0.220
0.0001	32	64	0.03	0.02	0.58	0.29	0.28	0.240
0.0001	32	128	0.08	0.03	0.56	0.28	0.30	0.250
0.0001	32	256	0.03	0.04	0.57	0.28	0.28	0.240
0.0001	64	32	0.05	0.01	0.54	0.27	0.28	0.230
0.0001	64	64	0.03	0.03	0.57	0.29	0.29	0.242
0.0001	64	128	0.02	0.03	0.58	0.27	0.30	0.240
0.0001	64	256	0.04	0.06	0.59	0.30	0.34	0.266
0.0001	128	32	0.04	0.01	0.61	0.26	0.29	0.242
0.0001	128	64	0.05	0.00	0.59	0.29	0.27	0.240
0.0001	128	128	0.03	0.01	0.56	0.27	0.25	0.224
0.0001	128	256	0.03	0.05	0.58	0.27	0.25	0.236
0.0002	32	32	0.08	0.01	0.58	0.26	0.33	0.252
0.0002	32	64	0.05	0.03	0.59	0.30	0.32	0.258
0.0002	32	128	0.06	0.04	0.57	0.25	0.29	0.242
0.0002	32	256	0.07	0.04	0.60	0.28	0.34	0.266
0.0002	64	32	0.03	0.02	0.56	0.27	0.30	0.236
0.0002	64	64	0.07	0.03	0.58	0.26	0.28	0.244
0.0002	64	128	0.07	0.06	0.55	0.29	0.25	0.244
0.0002	64	256	0.06	0.04	0.58	0.27	0.30	0.250
0.0002	128	32	0.00	0.03	0.59	0.27	0.28	0.234
0.0002	128	64	0.00	0.02	0.55	0.28	0.28	0.226
0.0002	128	128	0.05	0.04	0.55	0.27	0.30	0.242
0.0002	128	256	0.07	0.04	0.59	0.29	0.32	0.262
0.0005	32	32	0.05	0.02	0.55	0.31	0.29	0.244
0.0005	32	64	0.05	0.03	0.53	0.28	0.27	0.232
0.0005	32	128	0.06	0.04	0.54	0.26	0.27	0.234
0.0005	32	256	0.04	0.05	0.53	0.28	0.24	0.228
0.0005	64	32	0.06	0.04	0.56	0.26	0.29	0.242
0.0005	64	64	0.05	0.05	0.57	0.28	0.30	0.250
0.0005	64	128	0.06	0.04	0.57	0.29	0.27	0.246
0.0005	64	256	0.03	0.05	0.54	0.28	0.24	0.228
0.0005	128	32	0.06	0.03	0.57	0.28	0.26	0.240
0.0005	128	64	0.03	0.00	0.57	0.28	0.27	0.230
0.0005	128	128	0.06	0.05	0.56	0.27	0.35	0.258
0.0005	128	256	0.06	0.06	0.58	0.29	0.25	0.248

Table 13: Ablation results for Qwen2.5-7B-Instruct. LR denotes learning rate and BS stands for batch size. In each learning rate subgroup, the best performance is marked in bold.

LR	BS	Rank	AIME24	AIME25	MATH500	GPQA	AMC23	Avg
0.0001	32	32	0.18	0.12	0.79	0.36	0.50	0.390
0.0001	32	64	0.13	0.16	0.78	0.36	0.51	0.388
0.0001	32	128	0.17	0.11	0.78	0.37	0.55	0.396
0.0001	32	256	0.14	0.14	0.79	0.35	0.54	0.392
0.0001	64	32	0.14	0.12	0.77	0.35	0.54	0.384
0.0001	64	64	0.16	0.17	0.76	0.38	0.56	0.406
0.0001	64	128	0.15	0.15	0.78	0.39	0.55	0.404
0.0001	64	256	0.15	0.15	0.76	0.38	0.53	0.394
0.0001	128	32	0.12	0.12	0.77	0.37	0.55	0.386
0.0001	128	64	0.18	0.13	0.78	0.35	0.51	0.390
0.0001	128	128	0.19	0.11	0.78	0.36	0.52	0.392
0.0001	128	256	0.18	0.11	0.76	0.37	0.53	0.390
0.0002	32	32	0.12	0.14	0.77	0.36	0.54	0.386
0.0002	32	64	0.14	0.16	0.79	0.34	0.51	0.388
0.0002	32	128	0.18	0.18	0.78	0.35	0.54	0.406
0.0002	32	256	0.18	0.17	0.77	0.35	0.56	0.406
0.0002	64	32	0.21	0.11	0.79	0.36	0.52	0.398
0.0002	64	64	0.16	0.16	0.77	0.37	0.58	0.408
0.0002	64	128	0.20	0.14	0.80	0.38	0.52	0.408
0.0002	64	256	0.14	0.17	0.76	0.33	0.52	0.384
0.0002	128	32	0.15	0.11	0.77	0.35	0.57	0.390
0.0002	128	64	0.18	0.12	0.79	0.33	0.55	0.394
0.0002	128	128	0.18	0.15	0.77	0.35	0.57	0.404
0.0002	128	256	0.20	0.17	0.79	0.38	0.54	0.416
0.0005	32	32	0.17	0.15	0.76	0.38	0.53	0.398
0.0005	32	64	0.11	0.09	0.67	0.40	0.50	0.354
0.0005	32	128	0.16	0.11	0.76	0.37	0.47	0.374
0.0005	32	256	0.00	0.00	0.00	0.24	0.00	0.048
0.0005	64	32	0.17	0.14	0.76	0.38	0.54	0.398
0.0005	64	64	0.15	0.14	0.77	0.34	0.50	0.380
0.0005	64	128	0.22	0.12	0.79	0.37	0.59	0.418
0.0005	64	256	0.00	0.00	0.01	0.27	0.02	0.060
0.0005	128	32	0.19	0.13	0.79	0.34	0.55	0.400
0.0005	128	64	0.17	0.12	0.77	0.38	0.55	0.398
0.0005	128	128	0.23	0.17	0.78	0.37	0.53	0.416
0.0005	128	256	0.21	0.17	0.76	0.36	0.49	0.398

C Budget Forcing Details

Table 14 reports the list of hyperparameters we used in the budget forcing RL training.

Table 14: **GRPO Training Hyperparameters.** All experiments share these settings unless otherwise noted.

Hyperparameter	Value
Optimizer	AdamW
Learning Rate	2×10^{-5}
LR Scheduler	Cosine
Warmup Ratio	0.05
Batch Size (Global)	256
Generations per Prompt (G)	8
Temperature	0.8
Max Completion Length	6144
Max Gradient Norm	1.0
KL Penalty Coefficient (β_{KL})	$\{10^{-3}, 10^{-4}\}$
Training Steps	200

We report in Figures 9 and 10 additional qualitative comparisons between the reasoning traces of the unconstrained baseline against our budget-forced model.

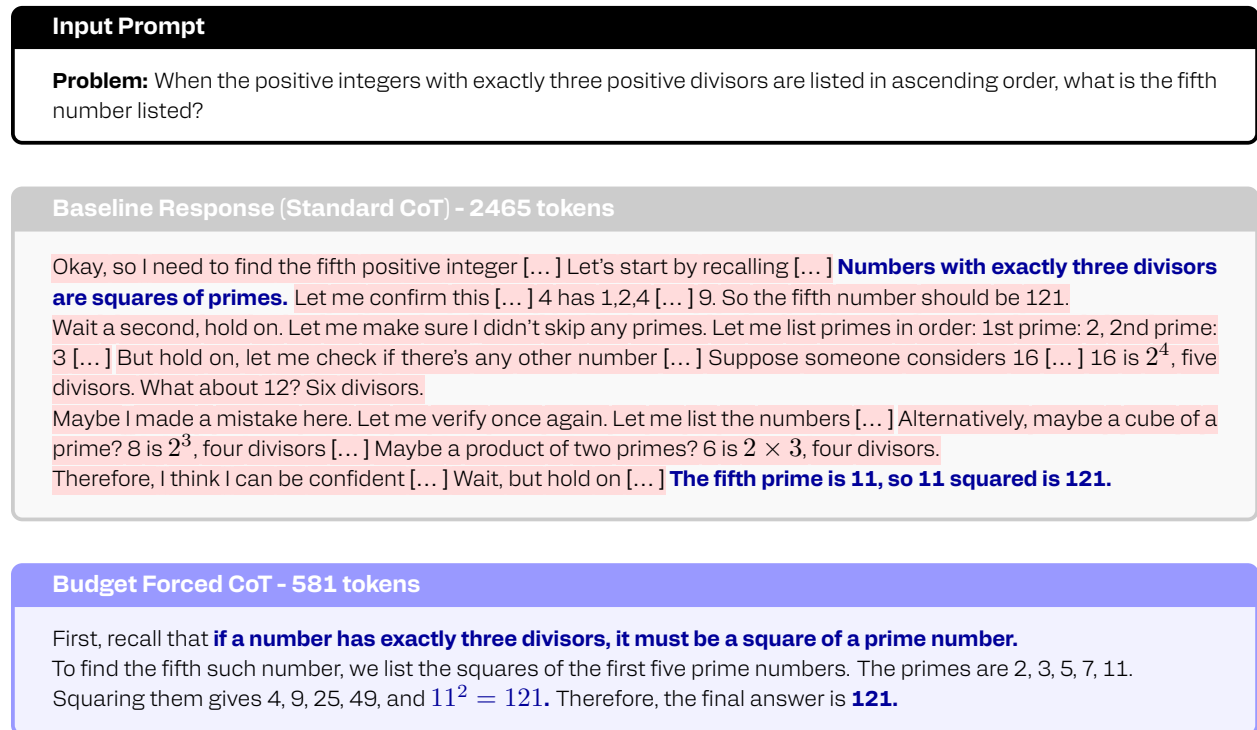


Figure 9: **Qualitative comparison on number theory reasoning.** Text highlighted in red denotes redundant verification and verbal parsing, while bold text identifies essential reasoning steps. We use “[...]” as a placeholder for brevity. **Top:** Prompt. **Middle:** The Baseline trace correctly identifies the property (p^2) early on but falls into extensive, redundant verification loops checking composite numbers and re-listing primes (highlighted in red). **Bottom:** The Budget Forced trace directly applies the prime-square property and computes the result without unnecessary hesitation or syntactic noise.

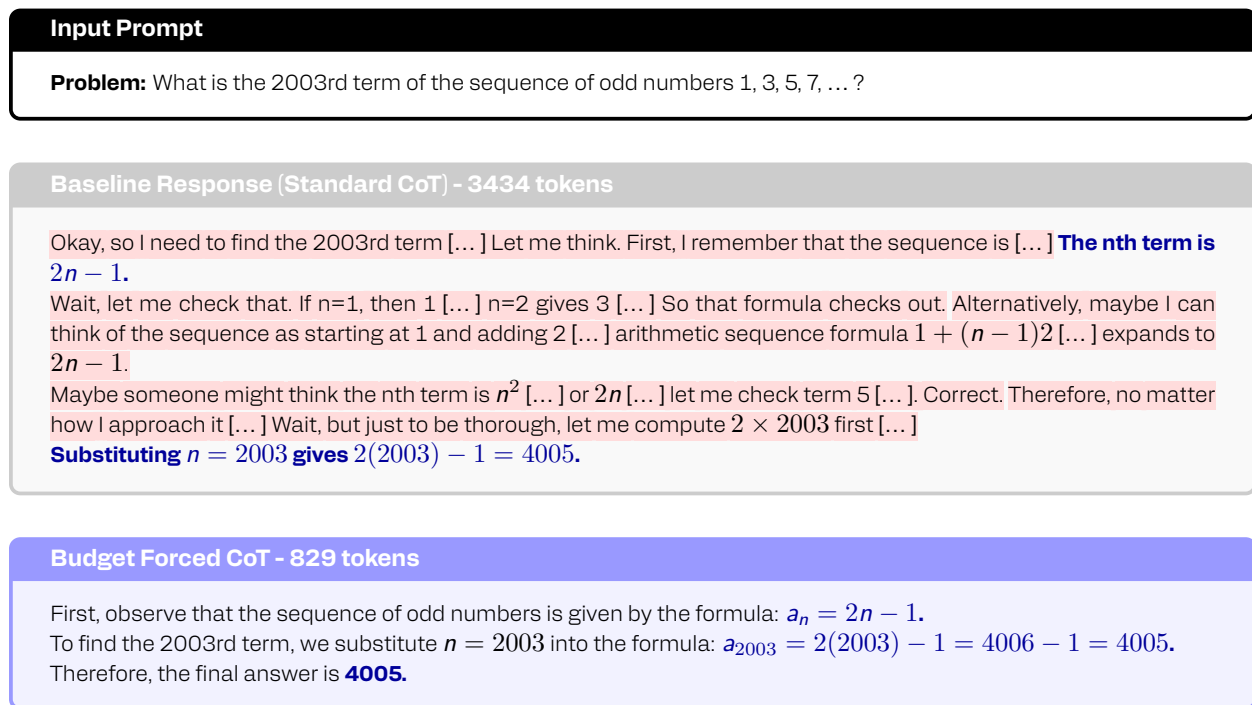


Figure 10: **Qualitative comparison on pattern recognition.** **Middle:** The Baseline trace correctly identifies the formula $2n - 1$ initially but spends nearly 1000 tokens validating it against alternative arithmetic formulas ($a + (n - 1)d$) and hypothetical user errors (confusing term number with value, testing n^2 , etc.). **Bottom:** The Budget Forced trace directly retrieves the formula and computes the specific term requested.